

Chapter 26

UART Controller (UART)

26.1 Overview

In embedded system applications, data are required to be transferred in a simple way with minimal system resources. This can be achieved by a Universal Asynchronous Receiver/Transmitter (UART), which flexibly exchanges data with other peripheral devices in full-duplex mode. ESP32-S3 has three UART controllers compatible with various UART devices. They support Infrared Data Association (IrDA) and RS485 transmission.

Each of the three UART controllers has a group of registers that function identically. In this chapter, the three UART controllers are referred to as UART n , in which n denotes 0, 1, or 2.

A UART is a character-oriented data link for asynchronous communication between devices. Such communication does not provide any clock signal to send data. Therefore, in order to communicate successfully, the transmitter and the receiver must operate at the same baud rate with the same stop bit(s) and parity bit.

A UART data frame usually begins with one start bit, followed by data bits, one parity bit (optional) and one or more stop bits. UART controllers on ESP32-S3 support various lengths of data bits and stop bits. These controllers also support software and hardware flow control as well as GDMA for seamless high-speed data transfer. This allows developers to use multiple UART ports at minimal software cost.

26.2 Features

Each UART controller has the following features:

- Three clock sources that can be divided
- Programmable baud rate
- 1024 x 8-bit RAM shared by TX FIFOs and RX FIFOs of the three UART controllers
- Full-duplex asynchronous communication
- Automatic baud rate detection of input signals
- Data bits ranging from 5 to 8
- Stop bits of 1, 1.5, or 2 bits
- Parity bit
- Special character AT_CMD detection
- RS485 protocol

- IrDA protocol
- High-speed data communication using GDMA
- UART as wake-up source
- Software and hardware flow control

26.3 UART Structure

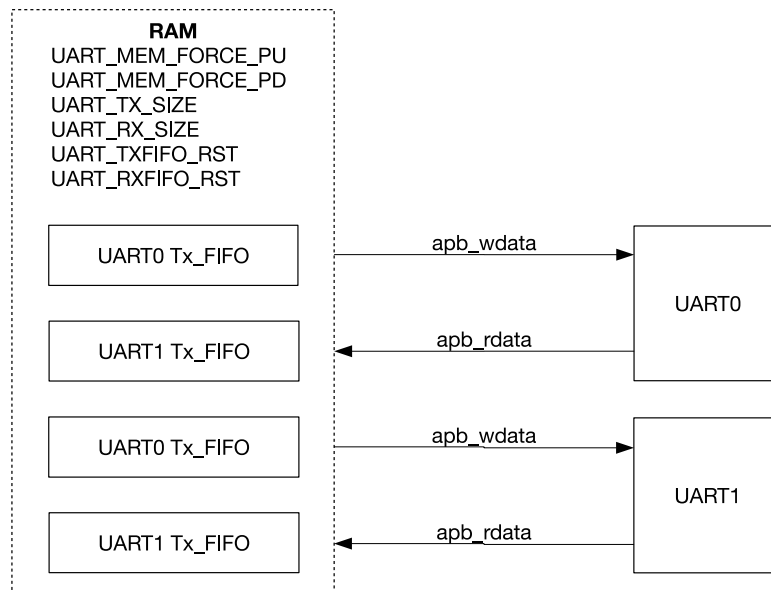


Figure 26.3-1. UART Architecture Overview

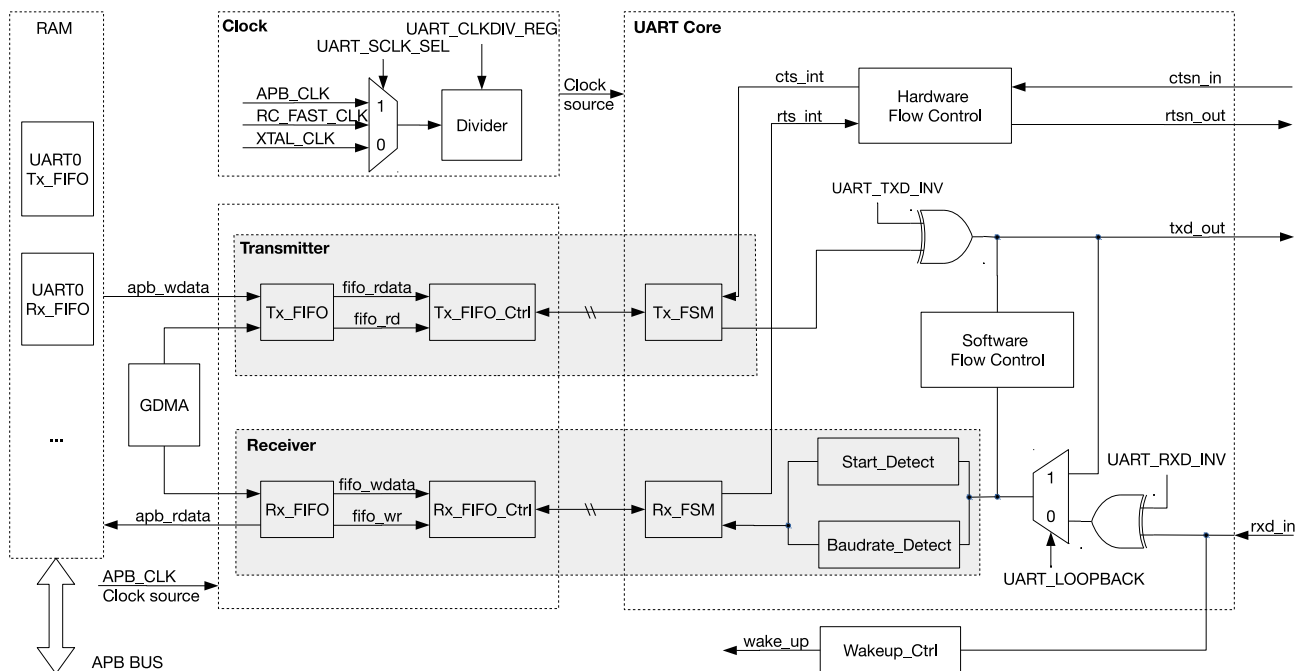


Figure 26.3-2. UART Structure

Figure 26.3-2 shows the basic structure of a UART controller. A UART controller works in two clock domains, namely APB_CLK domain and Core Clock domain (the UART Core's clock domain). The UART Core has three clock sources: 80 MHz APB_CLK, RC_FAST_CLK and external crystal clock XTAL_CLK (for details, please refer to Chapter 7 [Reset and Clock](#)), which are selected by configuring [UART_SCLK_SEL](#). The selected clock source is divided by a divider to generate clock signals that drive the UART Core.

A UART controller is broken down into two parts: a transmitter and a receiver.

The transmitter contains a FIFO, called TX FIFO (or Tx_FIFO), which buffers data to be sent. Software can write data to the Tx_FIFO either via the APB bus, or using GDMA. Tx_FIFO_Ctrl controls writing and reading the Tx_FIFO. When Tx_FIFO is not empty, Tx_FSM reads data bits in the data frame via Tx_FIFO_Ctrl, and converts them into a bitstream. The levels of output signal txd_out can be inverted by configuring the [UART_TXD_INV](#) field.

The receiver also contains a FIFO, called RX FIFO (or Rx_FIFO), which buffers received data. The levels of input signal rxd_in can be inverted by configuring [UART_RXD_INV](#) field. Baudrate_Detect measures the baud rate of input signal rxd_in by detecting its minimum pulse width. Start_Detect detects the start bit in a data frame. If the start bit is detected, Rx_FSM stores data bits in the data frame into Rx_FIFO by Rx_FIFO_Ctrl. Software can read data from Rx_FIFO via the APB bus, or receive data using GDMA.

HW_Flow_Ctrl controls rxd_in and txd_out data flows by standard UART RTS and CTS flow control signals (rtsn_out and ctsn_in). SW_Flow_Ctrl controls data flows by automatically adding special characters to outgoing data and detecting special characters in incoming data.

When a UART controller is in Light-sleep mode (see Chapter 10 [Low-power Management \(RTC_CNTL\)](#) for more details), Wakeup_Ctrl counts up rising edges of rxd_in. When the number is equal to or greater than [UART_ACTIVE_THRESHOLD](#) + 3, a wake_up signal is generated and sent to RTC, which then wakes up the ESP32-S3 chip.

26.4 Functional Description

26.4.1 Clock and Reset

UART controllers are asynchronous. their configuration registers, TX FIFOs, and RX FIFOs are in APB_CLK domain, while the module controlling transmission and reception (i.e., UART Core) is in Core Clock domain. The latter can be sourced out of three clocks, namely APB_CLK, RC_FAST_CLK and external crystal clock XTAL_CLK, which can be selected by configuring [UART_SCLK_SEL](#). The selected clock source can be divided. This divider supports fractional division, and the divisor is equal to:

$$UART_SCLK_DIV_NUM + \frac{UART_SCLK_DIV_B}{UART_SCLK_DIV_A}$$

The divisor ranges from 1 ~ 256.

When the frequency of the UART Core's clock is higher than the frequency needed to generate the baud rate, the UART Core can be clocked at a lower frequency by the divider, in order to reduce power consumption. Usually, the UART Core's clock frequency is lower than the APB_CLK's frequency, and can be divided by the largest divisor value when higher than the frequency needed to generate the baud rate. The frequency of the UART Core's clock can also be at most twice higher than the APB_CLK. The clock for the UART transmitter and the UART receiver can be controlled independently. To enable the clock for the UART transmitter,

[UART_TX_SCLK_EN](#) shall be set; to enable the clock for the UART receiver, [UART_RX_SCLK_EN](#) shall be set.

Section [26.5](#) explains the procedure to ensure that the configured register values are synchronized between APB_CLK domain and Core Clock domain.

Section [26.5.2.1](#) explains the procedure to reset the whole UART controller. Note that it is not recommended to only reset the APB clock domain module or UART Core.

26.4.2 UART RAM

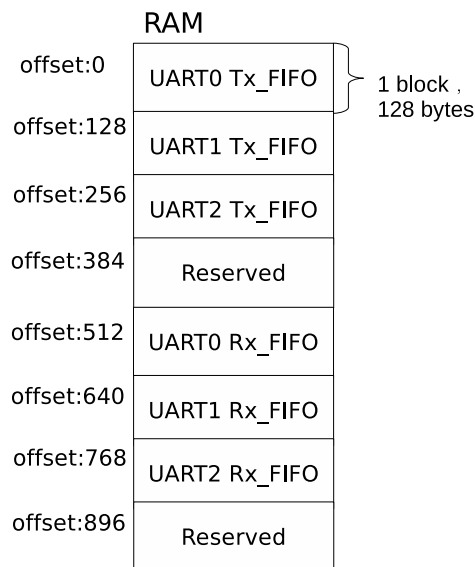


Figure 26.4-1. UART Controllers Sharing RAM

All three UART controllers on ESP32-S3 share 1024×8 bits of RAM. As Figure [26.4-1](#) illustrates, the RAM is divided into 8 blocks, each having 128×8 bits. Figure [26.4-1](#) shows how many RAM blocks are allocated by default to TX and RX FIFOs for each of the three UART controllers. UART n Tx_FIFO can be expanded by configuring [UART_TX_SIZE](#), while UART n Rx_FIFO can be expanded by configuring [UART_RX_SIZE](#). Some limits are imposed:

- UART0 Tx_FIFO can be increased up to 8 blocks (the whole RAM);
- UART1 Tx_FIFO can be increased up to 7 blocks (from offset 128 to the end address);
- UART2 Tx_FIFO can be increased up to 6 blocks (from offset 256 to the end address);
- UART0 Rx_FIFO can be increased up to 4 blocks (from offset 512 to the end address);
- UART1 Rx_FIFO can be increased up to 3 blocks (from offset 640 to the end address);
- UART2 Rx_FIFO can be increased up to 2 blocks (from offset 768 to the end address).

Please note that starting addresses of all FIFOs are fixed, so expanding one FIFO may take up the default space of other FIFOs. For example, by setting [UART_TX_SIZE](#) of UART0 to 2, the size of UART0 Tx_FIFO is increased by 128 bytes (from offset 0 to offset 255). In this case, UART0 Tx_FIFO takes up the default space for UART1 Tx_FIFO, and UART1's transmitting function cannot be used as a result.

When neither of the three UART controllers is active, RAM can enter low-power mode by setting [UART_MEM_FORCE_PD](#).

UART n Tx_FIFO is reset by setting [UART_TXFIFO_RST](#). UART n Rx_FIFO is reset by setting [UART_RXFIFO_RST](#).

The "empty" signal threshold for Tx_FIFO is configured by setting [UART_TXFIFO_EMPTY_THRHD](#). When data stored in Tx_FIFO is less than [UART_TXFIFO_EMPTY_THRHD](#), a UART_TXFIFO_EMPTY_INT interrupt is generated. The "full" signal threshold for Rx_FIFO is configured by setting [UART_RXFIFO_FULL_THRHD](#). When data stored in Rx_FIFO is equal to or greater than [UART_RXFIFO_FULL_THRHD](#), a UART_RXFIFO_FULL_INT interrupt is generated. In addition, when Rx_FIFO receives more data than its capacity, a UART_RXFIFO_OVF_INT interrupt is generated.

TX FIFO and RX FIFO can be accessed via the APB bus or GDMA. Access via the APB bus is performed through register [UART_FIFO_REG](#). You can put data into TX FIFO by writing [UART_RXFIFO_RD_BYTE](#), and get data in RX FIFO by reading this exact same field. For access via GDMA, please refer to Section 26.4.11.

26.4.3 Baud Rate Generation and Detection

26.4.3.1 Baud Rate Generation

Before a UART controller sends or receives data, the baud rate should be configured by setting corresponding registers. The baud rate generator of a UART controller functions by dividing the input clock source. It can divide the clock source by a fractional amount. The divisor is configured by [UART_CLKDIV_REG](#): [UART_CLKDIV](#) for the integer part, and [UART_CLKDIV_FRAG](#) for the fractional part. When using the 80 MHz input clock, the UART controller supports a maximum baud rate of 5 Mbaud.

The divisor of the baud rate is equal to

$$UART_CLKDIV + \frac{UART_CLKDIV_FRAG}{16}$$

meaning that the final baud rate is equal to

$$\frac{INPUT_FREQ}{UART_CLKDIV + \frac{UART_CLKDIV_FRAG}{16}}$$

where INPUT_FREQ is the frequency of UART Core's source clock. For example, if [UART_CLKDIV](#) = 694 and [UART_CLKDIV_FRAG](#) = 7 then the divisor value is

$$694 + \frac{7}{16} = 694.4375$$

When [UART_CLKDIV_FRAG](#) is 0, the baud rate generator is an integer clock divider where an output pulse is generated every [UART_CLKDIV](#) input pulses.

When [UART_CLKDIV_FRAG](#) is not 0, the divider is fractional and the output baud rate clock pulses are not strictly uniform. As shown in Figure 26.4-2, for every 16 output pulses, the frequency of some pulses is $INPUT_FREQ / (UART_CLKDIV + 1)$, and the frequency of the other pulses is $INPUT_FREQ / UART_CLKDIV$. A total of [UART_CLKDIV_FRAG](#) output pulses are generated by dividing $(UART_CLKDIV + 1)$ input pulses, and the remaining $(16 - UART_CLKDIV_FRAG)$ output pulses are generated by dividing [UART_CLKDIV](#) input pulses.

The output pulses are interleaved as shown in Figure 26.4-2 below, to make the output timing more uniform:

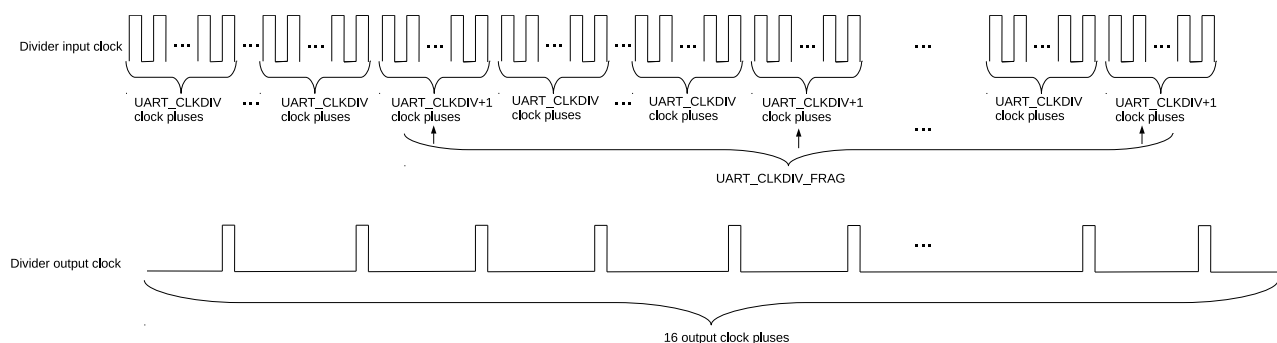


Figure 26.4-2. UART Controllers Division

To support IrDA (see Section 26.4.7 for details), the fractional clock divider for IrDA data transmission generates clock signals divided by $16 \times \text{UART_CLKDIV_REG}$. This divider works similarly as the one elaborated above: it takes $\text{UART_CLKDIV}/16$ as the integer value and the lowest four bits of UART_CLKDIV as the fractional value.

26.4.3.2 Baud Rate Detection

Automatic baud rate detection (Autobaud) on UARTs is enabled by setting `UART_AUTOBAUD_EN`. The Baudrate_Detect module shown in Figure 26.3-2 filters any noise whose pulse width is shorter than `UART_GLITCH_FILT`.

Before communication starts, the transmitter can send random data to the receiver for baud rate detection. `UART_LOWPULSE_MIN_CNT` stores the minimum low pulse width, `UART_HIGHPULSE_MIN_CNT` stores the minimum high pulse width, `UART_POSEDGE_MIN_CNT` stores the minimum pulse width between two rising edges, and `UART_NEGEDGE_MIN_CNT` stores the minimum pulse width between two falling edges. These four fields are read by software to determine the transmitter's baud rate.

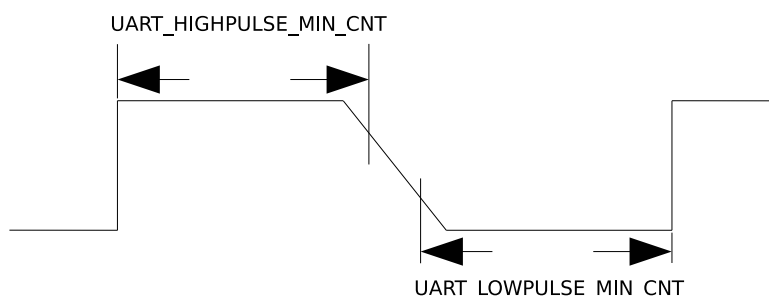


Figure 26.4-3. The Timing Diagram of Weak UART Signals Along Falling Edges

The baud rate can be determined in the following three ways:

1. Normally, to avoid sampling erroneous data along rising or falling edges in a metastable state, which results in the inaccuracy of `UART_LOWPULSE_MIN_CNT` or `UART_HIGHPULSE_MIN_CNT`, use a weighted average of these two values to eliminate errors. In this case, the baud rate is calculated as follows:

$$B_{\text{uart}} = \frac{f_{\text{clk}}}{(\text{UART_LOWPULSE_MIN_CNT} + \text{UART_HIGHPULSE_MIN_CNT} + 2)/2}$$

2. If UART signals are weak along falling edges as shown in Figure 26.4-3, which leads to an inaccurate average of `UART_LOWPULSE_MIN_CNT` and `UART_HIGHPULSE_MIN_CNT`, use

`UART_POSEDGE_MIN_CNT` to determine the transmitter's baud rate as follows:

$$B_{\text{uart}} = \frac{f_{\text{clk}}}{(\text{UART_POSEDGE_MIN_CNT} + 1)/2}$$

- If UART signals are weak along rising edges, use `UART_NEGEDGE_MIN_CNT` to determine the transmitter's baud rate as follows:

$$B_{\text{uart}} = \frac{f_{\text{clk}}}{(\text{UART_NEGEDGE_MIN_CNT} + 1)/2}$$

26.4.4 UART Data Frame

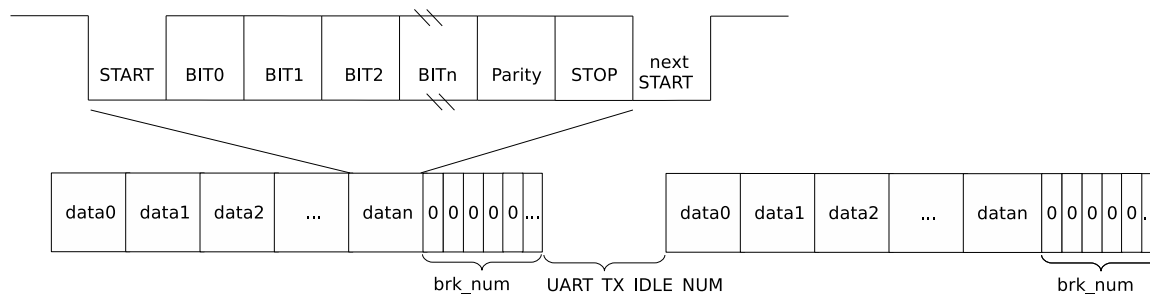


Figure 26.4-4. Structure of UART Data Frame

Figure 26.4-4 shows the basic structure of a data frame. A frame starts with one START bit, and ends with STOP bits which can be 1, 1.5, or 2 bits long, configured by `UART_STOP_BIT_NUM` (in RS485 mode turnaround delay may be added. See details in Section 26.4.6.2). The START bit is logical low, whereas STOP bits are logical high.

The actual data length can be anywhere between 5 ~ 8 bits, configured by `UART_BIT_NUM`. When `UART_PARITY_EN` is set, a parity bit is added after data bits. `UART_PARITY` is used to choose even parity or odd parity. When the receiver detects a parity bit error in the data received, a `UART_PARITY_ERR_INT` interrupt is generated, and the erroneous data are still stored into the RX FIFO. When the receiver detects a data frame error, a `UART_FRM_ERR_INT` interrupt is generated, and the erroneous data by default is stored into the RX FIFO.

If all data in `Tx_FIFO` have been sent, a `UART_TX_DONE_INT` interrupt is generated. After this, if the `UART_TXD_BRK` bit is set then the transmitter will enter the Break condition and send several NULL characters in which the TX data line is logical low. The number of NULL characters is configured by `UART_TX_BRK_NUM`. Once the transmitter has sent all NULL characters, a `UART_TX_BRK_DONE_INT` interrupt is generated. The minimum interval between data frames can be configured using `UART_TX_IDLE_NUM`. If the transmitter stays idle for `UART_TX_IDLE_NUM` or more time (in the unit of bit time, i.e., the time it takes to transfer one bit), a `UART_TX_BRK_IDLE_DONE_INT` interrupt is generated.

The receiver can also detect the Break conditions when the RX data line remains logical low for one NULL character transmission, and a `UART_BRK_DET_INT` interrupt will be triggered to detect that a Break condition has been completed.

The receiver can detect the current bus state through the timeout interrupt `UART_RXFIFO_TOUT_INT`. The `UART_RXFIFO_TOUT_INT` interrupt will be triggered when the bus is in the idle state for more than `UART_RX_TOUT_THRHD` bit time on current baud rate after the receiver has received at least one byte. You can use this interrupt to detect whether all the data from the transmitter has been sent.

26.4.5 AT_CMD Character Structure

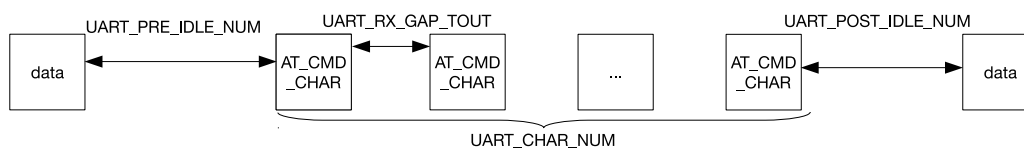


Figure 26.4-5. AT_CMD Character Structure

Figure 26.4-5 is the structure of a special character AT_CMD. If the receiver constantly receives AT_CMD_CHAR and the following conditions are met, a UART_AT_CMD_CHAR_DET_INT interrupt is generated. The specific value of AT_CMD_CHAR can be read from UART_n_AT_CMD_CHAR.

- The interval between the first AT_CMD_CHAR and the last non-AT_CMD_CHAR character is at least `UART_PRE_IDLE_NUM` cycles.
- The interval between two AT_CMD_CHAR characters is less than `UART_RX_GAP_TOUT` cycles.
- The number of AT_CMD_CHAR characters is equal to or greater than `UART_CHAR_NUM`.
- The interval between the last AT_CMD_CHAR character and next non-AT_CMD_CHAR character is at least `UART_POST_IDLE_NUM` cycles.

26.4.6 RS485

All three UART controllers support RS485 protocol. This protocol uses differential signals to transmit data, so it can communicate over longer distances at higher bit rates than RS232. RS485 has two-wire half-duplex mode and four-wire full-duplex modes. UART controllers support two-wire half-duplex transmission and bus snooping. In a two-wire RS485 multidrop network, there can be 32 slaves at most.

26.4.6.1 Driver Control

As shown in Figure 26.4-6, in a two-wire multidrop network, an external RS485 transceiver is needed for differential to single-ended conversion. An RS485 transceiver contains a driver and a receiver. When a UART controller is not in transmitter mode, the connection to the differential line can be broken by disabling the driver. When the DE (Driver Enable) signal is 1, the driver is enabled; when DE is 0, the driver is disabled.

The UART receiver converts differential signals to single-ended signals via an external receiver. RE is the enable control signal for the receiver. When RE is 0, the receiver is enabled; when RE is 1, the receiver is disabled. If RE is configured as 0, the UART controller is allowed to snoop data on the bus, including the data sent by itself.

DE can be controlled by either software or hardware. To reduce the cost of software, DE is controlled by hardware in our design. As shown in Figure 26.4-6, DE is connected to dtrn_out of UART (please refer to Section 26.4.10.1 for more details).

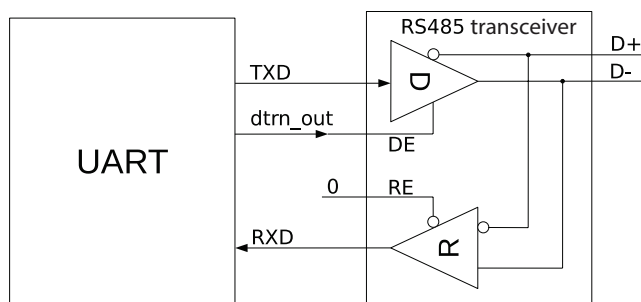


Figure 26.4-6. Driver Control Diagram in RS485 Mode

26.4.6.2 Turnaround Delay

By default, all three UART controllers work in receiver mode. When a UART controller is switched from transmitter mode to receiver mode, the RS485 protocol requires a turnaround delay of one cycle after the stop bit. The UART transmitter supports adding a turnaround delay of one cycle not only before the start bit but also after the stop bit. When `UART_DLO_EN` is set, a turnaround delay of one cycle is added before the start bit; when `UART_DL1_EN` is set, a turnaround delay of one cycle is added after the stop bit.

26.4.6.3 Bus Snooping

In a two-wire multidrop network, UART controllers support bus snooping if RE of the external RS485 transceiver is 0. By default, a UART controller is not allowed to transmit and receive data simultaneously. If `UART_RS485TX_RX_EN` is set and the external RS485 transceiver is configured as in Figure 26.4-6, a UART controller may receive data in transmitter mode and snoop the bus. If `UART_RS485RXBY_TX_EN` is set, a UART controller may transmit data in receiver mode.

All three UART controllers can snoop the data sent by themselves. In transmitter mode, when a UART controller monitors a collision between the data sent and the data received, a `UART_RS485_CLASH_INT` interrupt is generated; when it monitors a data frame error, a `UART_RS485_FRM_ERR_INT` interrupt is generated; when it monitors a polarity error, a `UART_RS485_PARITY_ERR_INT` is generated.

26.4.7 IrDA

IrDA protocol consists of three layers, namely the physical layer, the link access protocol, and the link management protocol. The three UART controllers implement IrDA's physical layer. In IrDA encoding, a UART controller supports data rates up to 115.2 kbit/s (SIR, or serial infrared mode). As shown in Figure 26.4-7, the IrDA encoder converts a NRZ (non-return to zero code) signal to a RZI (return to zero inverted code) signal and sends it to the external driver and infrared LED. This encoder uses modulated signals whose pulse width is 3/16 bits to indicate logic "0", and low levels to indicate logic "1". The IrDA decoder receives signals from the infrared receiver and converts them to NRZ signals. In most cases, the receiver is high when it is idle, and the encoder output polarity is the opposite of the decoder input polarity. If a low pulse is detected, it indicates that a start bit has been received.

When IrDA function is enabled, one bit is divided into 16 clock cycles. If the bit to be sent is zero, then the 9th, 10th and 11th clock cycle are high.

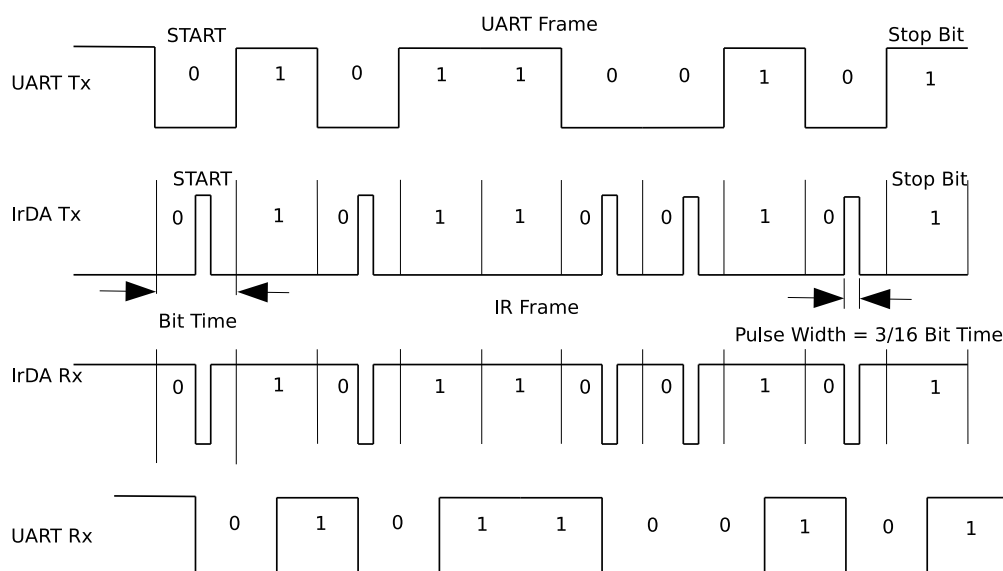


Figure 26.4-7. The Timing Diagram of Encoding and Decoding in SIR mode

The IrDA transceiver is half-duplex, meaning that it cannot send and receive data simultaneously. As shown in Figure 26.4-8, IrDA function is enabled by setting `UART_IRDA_EN`. When `UART_IRDA_TX_EN` is set (high), the IrDA transceiver is enabled to send data and not allowed to receive data; when `UART_IRDA_TX_EN` is reset (low), the IrDA transceiver is enabled to receive data and not allowed to send data.

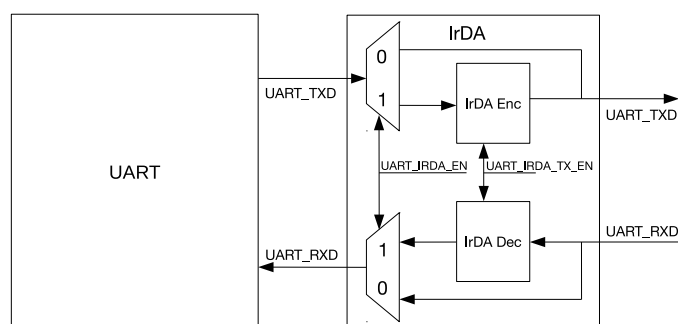


Figure 26.4-8. IrDA Encoding and Decoding Diagram

26.4.8 Wake-up

UART0 and UART1 can be set as a wake-up source for Light-sleep mode. To be specific Wakeup_Ctrl counts up the rising edges of rxd_in, and when this count is equal to or greater than `UART_ACTIVE_THRESHOLD + 3`, a wake_up signal is generated and sent to RTC, which then wakes ESP32-S3 up.

After the chip is woken up by UART, it is necessary to clear the wake_up signal by transmitting data to UART in Active mode or resetting the whole UART, otherwise the number of rising edges required for the next wakeup will be reduced.

26.4.9 Loopback Test

UART_n supports loopback testing, which can be enabled by setting `UART_LOOPBACK`. When loopback testing is enabled, UART output signal txd_out is connected to its input signal rxd_in, rtsn_out is connected to

ctsn_in, and dtrn_out is connected to dsrn_out. Data are then sent out through txd_out. If the data received match the data sent, it indicates that UART_n controller is working properly.

26.4.10 Flow Control

UART controllers have two ways to control data flow, namely hardware flow control and software flow control. Hardware flow control is achieved using output signal rtsn_out and input signal dsrn_in. Software flow control is achieved by inserting special characters (XON or XOFF) in the data flow sent and detecting special characters in the data flow received.

26.4.10.1 Hardware Flow Control

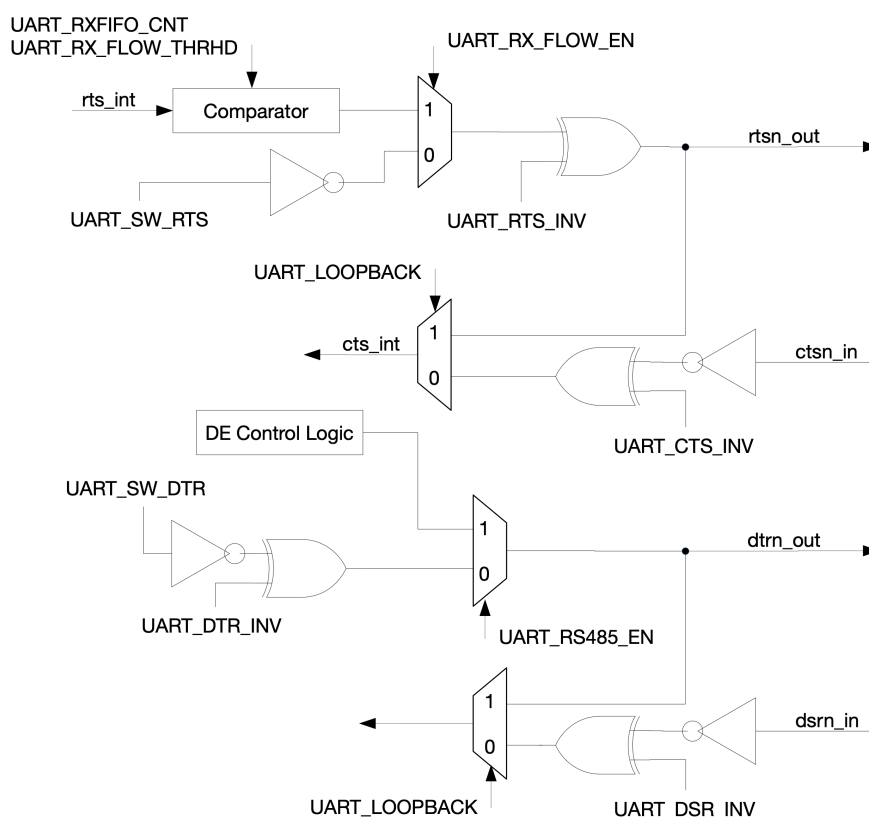


Figure 26.4-9. Hardware Flow Control Diagram

Figure 26.4-9 shows the hardware flow control of a UART controller. Hardware flow control uses output signal rtsn_out and input signal dsrn_in. Figure 26.4-10 illustrates how these signals are connected between UART on ESP32-S3 (hereinafter referred to as IUO) and the external UART (hereinafter referred to as EUO).

When rtsn_out of IUO is low, EUO is allowed to send data; when rtsn_out of IUO is high, EUO is notified to stop sending data until rtsn_out of IUO returns to low. The output signal rtsn_out can be controlled in two ways.

- Software control: Enter this mode by clearing `UART_RX_FLOW_EN` to 0. In this mode, the level of rtsn_out is changed by configuring `UART_SW_RTS`.
- Hardware control: Enter this mode by setting `UART_RX_FLOW_EN` to 1. In this mode, rtsn_out is pulled high when data in Rx_FIFO exceeds `UART_RX_FLOW_THRHD`.

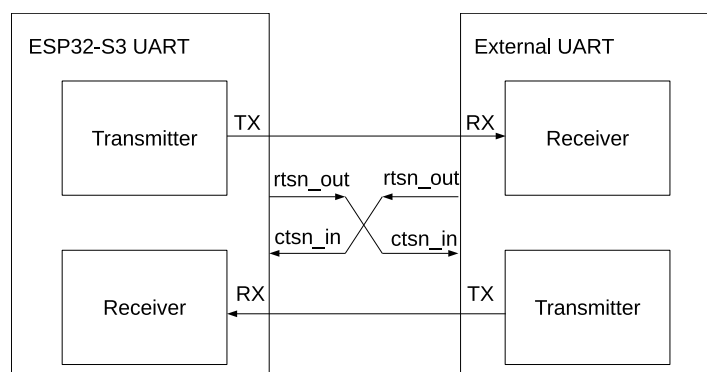


Figure 26.4-10. Connection between Hardware Flow Control Signals

When `ctsn_in` of IUO is low, IUO is allowed to send data; when `ctsn_in` is high, IUO is not allowed to send data. When IUO detects an edge change on `ctsn_in`, a `UART_CTS_CHG_INT` interrupt is generated.

If `dtrn_out` of IUO is high, it indicates that IUO is ready to transmit data. `dtrn_out` is generated by configuring the `UART_SW_DTR` field. When the IUO transmitter detects a edge change on `dtrn_in`, a `UART_DSR_CHG_INT` interrupt is generated. After this interrupt is detected, software can obtain the level of input signal `dtrn_in` by reading `UART_DSRN`. If `dtrn_in` is high, it indicates that EUO is ready to transmit data.

In a two-wire RS485 multidrop network enabled by setting `UART_RS485_EN`, `dtrn_out` is generated by hardware and used for transmit/receive turnaround. When data transmission starts, `dtrn_out` is pulled high and the external driver is enabled; when data transmission completes, `dtrn_out` is pulled low and the external driver is disabled. Please note that when there is a turnaround delay of one cycle added after the stop bit, `dtrn_out` is pulled low after the delay.

26.4.10.2 Software Flow Control

Instead of CTS/RTS lines, software flow control uses XON/XOFF characters to start or stop data transmission. Such flow control can be enabled by setting `UART_SW_FLOW_CON_EN` to 1.

When choosing software flow control, the hardware automatically detects if XON and XOFF characters are used in the data flow, and generates a `UART_SW_XOFF_INT` or a `UART_SW_XON_INT` interrupt accordingly. When XOFF character is detected, the transmitter stops data transmission once the current byte has been transmitted; when XON character is detected, the transmitter starts data transmission. In addition, software can force the transmitter to stop sending data or to start sending data by setting respectively `UART_FORCE_XOFF` or `UART_FORCE_XON`.

Software determines whether to insert flow control characters according to the remaining room in the RX FIFO. When `UART_SEND_XOFF` is set, the transmitter sends an XOFF character configured by `UART_XOFF_CHAR` after the current byte in transmission; when `UART_SEND_XON` is set, the transmitter sends an XON character configured by `UART_XON_CHAR` after the current byte in transmission. If the RX FIFO of a UART controller stores more data than `UART_XOFF_THRESHOLD`, `UART_SEND_XOFF` is set by hardware. As a result, the transmitter sends an XOFF character configured by `UART_XOFF_CHAR` after the current byte in transmission. If the RX FIFO of a UART controller stores less data than `UART_XON_THRESHOLD`, `UART_SEND_XON` is set by hardware. As a result, the transmitter sends an XON character configured by `UART_XON_CHAR` after the current byte in transmission.

26.4.11 GDMA Mode

All three UART controllers on ESP32-S3 share one TX/RX GDMA (general direct memory access) channel via UHCI. In GDMA mode, UART controllers support the decoding and encoding of HCI data packets. The `UHCI_UART $n$ _CE` field determines which UART controller occupies the GDMA TX/RX channel.

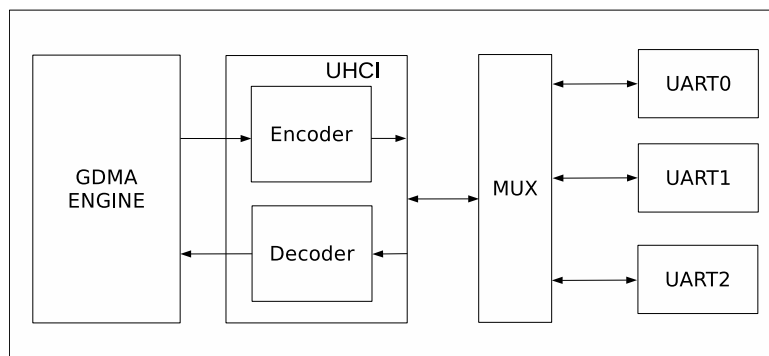


Figure 26.4-11. Data Transfer in GDMA Mode

Figure 26.4-11 shows how data are transferred using GDMA. Before GDMA receives data, software prepares an inlink (i.e., a linked list of receive descriptors. For details, see Chapter 3 [GDMA Controller \(GDMA\)](#)).

`GDMA_INLINK_ADDR_CH $n$` points to the first receive descriptor in the inlink. After `GDMA_INLINK_START_CH $n$` is set, UHCI passes data that UART has received to the decoder. The decoded data are then stored into the RAM pointed by the inlink under the control of GDMA.

Before GDMA sends data, software prepares an outlink and data to be sent. `GDMA_OUTLINK_ADDR_CH $n$` points to the first transmit descriptor in the outlink. After `GDMA_OUTLINK_START_CH $n$` is set, GDMA reads data from the RAM pointed by outlink. The data are then encoded by the encoder, and sent sequentially by the UART transmitter.

HCI data packets have separators at the beginning and the end, with data bits in the middle (separators + data bits + separators). The encoder inserts separators in front of and after data bits, and replaces data bits identical to separators with special characters (i.e., escape characters). The decoder removes separators in front of and after data bits, and replaces escape characters with separators. There can be more than one continuous separator at the beginning and the end of a data packet. The separator is configured by `UHCI_SEPER_CHAR`, 0xC0 by default. The escape characters are configured by `UHCI_ESC_SEQO_CHAR0` (0xDB by default) and `UHCI_ESC_SEQO_CHAR1` (0xDD by default). When all data have been sent, a `GDMA_OUT_TOTAL_EOF_CH $n$ _INT` interrupt is generated. When all data have been received, a `GDMA_IN_SUC_EOF_CH $n$ _INT` is generated.

26.4.12 UART Interrupts

- `UART_AT_CMD_CHAR_DET_INT`: Triggered when the receiver detects an AT_CMD character.
- `UART_RS485_CLASH_INT`: Triggered when a collision is detected between the transmitter and the receiver in RS485 mode.
- `UART_RS485_FRM_ERR_INT`: Triggered when an error is detected in the data frame sent by the transmitter in RS485 mode.

- `UART_RS485_PARITY_ERR_INT`: Triggered when an error is detected in the parity bit sent by the transmitter in RS485 mode.
- `UART_TX_DONE_INT`: Triggered when all data in the TX FIFO have been sent.
- `UART_TX_BRK_IDLE_DONE_INT`: Triggered when the transmitter stays idle after sending the last data bit. The minimum amount of time marking the transmitter state as idle is determined by the configurable threshold value.
- `UART_TX_BRK_DONE_INT`: Triggered when the transmitter has sent all NULL characters following the complete transmission of data from the TX FIFO.
- `UART_GLITCH_DET_INT`: Triggered when the receiver detects a glitch in the middle of the start bit.
- `UART_SW_XOFF_INT`: Triggered when `UART_SW_FLOW_CON_EN` is set and the receiver receives a XOFF character.
- `UART_SW_XON_INT`: Triggered when `UART_SW_FLOW_CON_EN` is set and the receiver receives a XON character.
- `UART_RXFIFO_TOUT_INT`: Triggered when the receiver takes more time than `UART_RX_TOUT_THRHD` to receive one byte.
- `UART_BRK_DET_INT`: Triggered when the receiver detects a NULL character (i.e., logic 0 for one NULL character transmission) after stop bits.
- `UART_CTS_CHG_INT`: Triggered when the receiver detects an edge change on CTSn signals.
- `UART_DSR_CHG_INT`: Triggered when the receiver detects an edge change on DSRn signals.
- `UART_RXFIFO_OVF_INT`: Triggered when the receiver receives more data than the capacity of the RX FIFO.
- `UART_FRM_ERR_INT`: Triggered when the receiver detects a data frame error.
- `UART_PARITY_ERR_INT`: Triggered when the receiver detects a parity error.
- `UART_TXFIFO_EMPTY_INT`: Triggered when the TX FIFO stores less data than what `UART_TXFIFO_EMPTY_THRHD` specifies.
- `UART_RXFIFO_FULL_INT`: Triggered when the receiver receives more data than what `UART_RXFIFO_FULL_THRHD` specifies.
- `UART_WAKEUP_INT`: Triggered when UART is woken up.

26.4.13 UHCI Interrupts

- `UHCI_APP_CTRL1_INT`: Triggered when software sets `UHCI_APP_CTRL1_INT_RAW`.
- `UHCI_APP_CTRL0_INT`: Triggered when software sets `UHCI_APP_CTRL0_INT_RAW`.
- `UHCI_OUTLINK_EOF_ERR_INT`: Triggered when an EOF error is detected in a transmit descriptor.
- `UHCI_SEND_A_REG_Q_INT`: Triggered when UHCI has sent a series of short packets using `always_send`.
- `UHCI_SEND_S_REG_Q_INT`: Triggered when UHCI has sent a series of short packets using `single_send`.
- `UHCI_TX_HUNG_INT`: Triggered when UHCI takes too long to read RAM using a GDMA transmit channel.
- `UHCI_RX_HUNG_INT`: Triggered when UHCI takes too long to receive data using a GDMA receive channel.

- UHCI_TX_START_INT: Triggered when GDMA detects a separator character.
- UHCI_RX_START_INT: Triggered when a separator character has been sent.

26.5 Programming Procedures

26.5.1 Register Type

All UART registers are in APB_CLK domain. According to whether clock domain crossing and synchronization are required, UART registers that can be configured by software are classified into three types, namely synchronous registers, static registers, and immediate registers. Synchronous registers are read in Core Clock domain, and take effect after synchronization. Static registers are also read in Core Clock domain, but would not change dynamically. Therefore, for static registers, clock domain crossing is not required, and software can turn on and off the clock for the UART transmitter or receiver to ensure that the configuration sampled in Core Clock domain is correct. Immediate registers are read in APB_CLK domain, and take effect after being configured via the APB bus.

26.5.1.1 Synchronous Registers

Since synchronous registers are read in core clock domain, but written in APB_CLK domain, they implement the clock domain crossing design to ensure that their values sampled in Core Clock domain are correct.

These registers as listed in Table 26.5-1 are configured as follows:

- Enable register synchronization by clearing [UART_UPDATE_CTRL](#) to 0;
- Wait for [UART_REG_UPDATE](#) to become 0, which indicates the completion of last synchronization;
- Configure synchronous registers;
- Synchronize the configured values to Core Clock domain by writing 1 to [UART_REG_UPDATE](#).

Table 26.5-1. UART_n Synchronous Registers

Register	Field
UART_CLKDIV_REG	UART_CLKDIV_FRAG[3:0]
	UART_CLKDIV[11:0]
UART_CONFO_REG	UART_AUTOBAUD_EN
	UART_ERR_WR_MASK
	UART_TXD_INV
	UART_RXD_INV
	UART_IRDA_EN
	UART_TX_FLOW_EN
	UART_LOOPBACK
	UART_IRDA_RX_INV
	UART_IRDA_TX_EN
	UART_IRDA_WCTL
	UART_IRDA_TX_EN
	UART_IRDA_DPLX
	UART_STOP_BIT_NUM

Cont'd on next page

Table 26.5-1 – cont'd from previous page

Register	Field
	UART_BIT_NUM
	UART_PARITY_EN
	UART_PARITY

Cont'd on next page

Table 26.5-1 – cont'd from previous page

Register	Field
UART_FLOW_CONF_REG	UART_SEND_XOFF
	UART_SEND_XON
	UART_FORCE_XOFF
	UART_FORCE_XON
	UART_XONOFF_DEL
	UART_SW_FLOW_CON_EN
UART_RS485_CONF_REG	UART_RS485_TX_DLY_NUM[3:0]
	UART_RS485_RX_DLY_NUM
	UART_RS485RXBY_TX_EN
	UART_RS485TX_RX_EN
	UART_DL1_EN
	UART_DLO_EN
	UART_RS485_EN

26.5.1.2 Static Registers

Static registers, though also read in Core Clock domain, would not change dynamically when UART controllers are at work, so they do not implement the clock domain crossing design. These registers must be configured when the UART transmitter or receiver is not at work. In this case, software can turn off the clock for the UART transmitter or receiver, so that static registers are not sampled in their metastable state. When software turns on the clock, the configured values are stable to be correctly sampled. Static registers as listed in Table 26.5-2 are configured as follows:

- Turn off the clock for the UART transmitter by clearing [UART_TX_SCLK_EN](#), or the clock for the UART receiver by clearing [UART_RX_SCLK_EN](#), depending on which one (transmitter or receiver) is not at work;
- Configure static registers;
- Turn on the clock for the UART transmitter by writing 1 to [UART_TX_SCLK_EN](#), or the clock for the UART receiver by writing 1 to [UART_RX_SCLK_EN](#).

Table 26.5-2. UART_n Static Registers

Register	Field
UART_RX_FILT_REG	UART_GLITCH_FILT_EN
	UART_GLITCH_FILT[7:0]
UART_SLEEP_CONF_REG	UART_ACTIVE_THRESHOLD[9:0]
UART_SWFC_CONFO_REG	UART_XOFF_CHAR[7:0]
UART_SWFC_CONF1_REG	UART_XON_CHAR[7:0]
UART_IDLE_CONF_REG	UART_TX_IDLE_NUM[9:0]
UART_AT_CMD_PRECNT_REG	UART_PRE_IDLE_NUM[15:0]
UART_AT_CMD_POSTCNT_REG	UART_POST_IDLE_NUM[15:0]
UART_AT_CMD_GAP_TOUT_REG	UART_RX_GAP_TOUT[15:0]
UART_AT_CMD_CHAR_REG	UART_CHAR_NUM[7:0]

Cont'd on next page

Table 26.5-2 – cont'd from previous page

Register	Field
	UART_AT_CMD_CHAR[7:0]

26.5.1.3 Immediate Registers

Except those listed in Table 26.5-1 and Table 26.5-2, registers that can be configured by software are immediate registers read in APB_CLK domain, such as interrupt and FIFO configuration registers.

26.5.2 Detailed Steps

Figure 26.5-1 illustrates the process to program UART controllers, namely initializing the UART, configuring the registers, enabling the transmitter and/or receiver, and finishing data transmission.

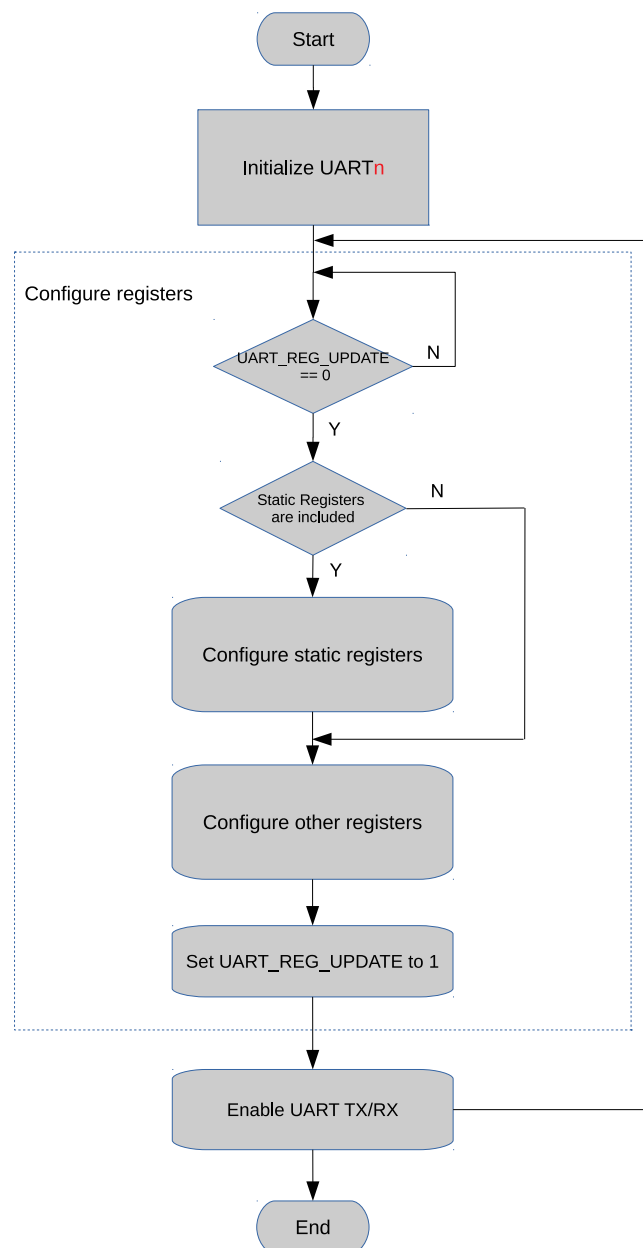


Figure 26.5-1. UART Programming Procedures

26.5.2.1 Initializing UART n

Initializing UART n requires two steps: resetting UART n and enabling register synchronization.

To reset UART n :

- enable the clock for UART RAM by setting `SYSTEM_UART_MEM_CLK_EN` to 1;
- enable APB_CLK for UART n by setting `SYSTEM_UART $n$ _CLK_EN` to 1;
- clear `SYSTEM_UART $n$ _RST`;
- write 1 to `UART_RST_CORE`;
- write 1 to `SYSTEM_UART $n$ _RST`;
- clear `SYSTEM_UART $n$ _RST`;
- clear `UART_RST_CORE`.

To enable register synchronization, clear `UART_UPDATE_CTRL`.

26.5.2.2 Configuring UART n Communication

To configure UART n communication:

- wait for `UART_REG_UPDATE` to become 0, which indicates the completion of the last synchronization;
- configure static registers (if any) following Section 26.5.1.2;
- select the clock source via `UART_SCLK_SEL`;
- configure divisor of the divider via `UART_SCLK_DIV_NUM`, `UART_SCLK_DIV_A`, and `UART_SCLK_DIV_B`;
- configure the baud rate for transmission via `UART_CLKDIV` and `UART_CLKDIV_FRAG`;
- configure data length via `UART_BIT_NUM`;
- configure odd or even parity check via `UART_PARITY_EN` and `UART_PARITY`;
- optional steps depending on application ...
- synchronize the configured values to the Core Clock domain by writing 1 to `UART_REG_UPDATE`.

26.5.2.3 Enabling UART n

To enable UART n transmitter:

- configure the TX FIFO's empty threshold via `UART_TXFIFO_EMPTY_THRHD`;
- disable `UART_TXFIFO_EMPTY_INT` interrupt by clearing `UART_TXFIFO_EMPTY_INT_ENA`;
- write data to be sent to `UART_RXFIFO_RD_BYTE`;
- clear `UART_TXFIFO_EMPTY_INT` interrupt by setting `UART_TXFIFO_EMPTY_INT_CLR`;
- enable `UART_TXFIFO_EMPTY_INT` interrupt by setting `UART_TXFIFO_EMPTY_INT_ENA`;
- detect `UART_TXFIFO_EMPTY_INT` and wait for the completion of data transmission.

To enable UART n receiver:

- configure RXFIFO's full threshold via `UART_RXFIFO_FULL_THRHD`;

- enable UART_RXFIFO_FULL_INT interrupt by setting [UART_RXFIFO_FULL_INT_ENA](#);
- detect UART_TXFIFO_FULL_INT and wait until the RXFIFO is full;
- read data from RXFIFO via [UART_RXFIFO_RD_BYTE](#), and obtain the number of bytes received in RXFIFO via [UART_RXFIFO_CNT](#).

26.6 Register Summary

26.6.1 UART Register Summary

The addresses in this section are relative to **UART Controller** base address provided in Table 4.3-3 in Chapter 4 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
FIFO Configuration			
UART_FIFO_REG	FIFO data register	0x0000	RO
UART_MEM_CONF_REG	UART threshold and allocation configuration	0x0060	R/W
UART Interrupt Register			
UART_INT_RAW_REG	Raw interrupt status	0x0004	R/WTC/SS
UART_INT_ST_REG	Masked interrupt status	0x0008	RO
UART_INT_ENA_REG	Interrupt enable bits	0x000C	R/W
UART_INT_CLR_REG	Interrupt clear bits	0x0010	WT
Configuration Register			
UART_CLKDIV_REG	Clock divider configuration	0x0014	R/W
UART_RX_FILT_REG	RX filter configuration	0x0018	R/W
UART_CONFO_REG	Configuration register 0	0x0020	R/W
UART_CONF1_REG	Configuration register 1	0x0024	R/W
UART_FLOW_CONF_REG	Software flow control configuration	0x0034	varies
UART_SLEEP_CONF_REG	Sleep mode configuration	0x0038	R/W
UART_SWFC_CONFO_REG	Software flow control character configuration	0x003C	R/W
UART_SWFC_CONF1_REG	Software flow control character configuration	0x0040	R/W
UART_TXBRK_CONF_REG	TX break character configuration	0x0044	R/W
UART_IDLE_CONF_REG	Frame end idle time configuration	0x0048	R/W
UART_RS485_CONF_REG	RS485 mode configuration	0x004C	R/W
UART_CLK_CONF_REG	UART core clock configuration	0x0078	R/W
Status Register			
UART_STATUS_REG	UART status register	0x001C	RO
UART_MEM_TX_STATUS_REG	TX FIFO write and read offset address	0x0064	RO
UART_MEM_RX_STATUS_REG	RX FIFO write and read offset address	0x0068	RO
UART_FSM_STATUS_REG	UART transmitter and receiver status	0x006C	RO
Autobaud Register			
UART_LOWPULSE_REG	Autobaud minimum low pulse duration register	0x0028	RO
UART_HIGHPULSE_REG	Autobaud minimum high pulse duration register	0x002C	RO
UART_RXD_CNT_REG	Autobaud edge change count register	0x0030	RO
UART_POSPULSE_REG	Autobaud high pulse register	0x0070	RO
UART_NEGPULSE_REG	Autobaud low pulse register	0x0074	RO
AT Escape Sequence Selection Configuration			

Name	Description	Address	Access
UART_AT_CMD_PRECNT_REG	Pre-sequence timing configuration	0x0050	R/W
UART_AT_CMD_POSTCNT_REG	Post-sequence timing configuration	0x0054	R/W
UART_AT_CMD_GAPTOU_REG	Timeout configuration	0x0058	R/W
UART_AT_CMD_CHAR_REG	AT escape sequence detection configuration	0x005C	R/W
Version Register			
UART_DATE_REG	UART version control register	0x007C	R/W
UART_ID_REG	UART ID register	0x0080	varies

26.6.2 UHCI Register Summary

The addresses in this section are relative to **UHCI Controller** base address provided in Table 4.3-3 in Chapter 4 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
Configuration Register			
UHCI_CONFO_REG	UHCI configuration register	0x0000	R/W
UHCI_CONF1_REG	UHCI configuration register	0x0018	varies
UHCI_ESCAPE_CONF_REG	Escape character configuration	0x0024	R/W
UHCI_HUNG_CONF_REG	Timeout configuration	0x0028	R/W
UHCI_ACK_NUM_REG	UHCI ACK number configuration	0x002C	varies
UHCI_QUICK_SENT_REG	UHCI quick_sent configuration register	0x0034	varies
UHCI_REG_Q0_WORD0_REG	Q0_WORD0 quick_sent register	0x0038	R/W
UHCI_REG_Q0_WORD1_REG	Q0_WORD1 quick_sent register	0x003C	R/W
UHCI_REG_Q1_WORD0_REG	Q1_WORD0 quick_sent register	0x0040	R/W
UHCI_REG_Q1_WORD1_REG	Q1_WORD1 quick_sent register	0x0044	R/W
UHCI_REG_Q2_WORD0_REG	Q2_WORD0 quick_sent register	0x0048	R/W
UHCI_REG_Q2_WORD1_REG	Q2_WORD1 quick_sent register	0x004C	R/W
UHCI_REG_Q3_WORD0_REG	Q3_WORD0 quick_sent register	0x0050	R/W
UHCI_REG_Q3_WORD1_REG	Q3_WORD1 quick_sent register	0x0054	R/W
UHCI_REG_Q4_WORD0_REG	Q4_WORD0 quick_sent register	0x0058	R/W
UHCI_REG_Q4_WORD1_REG	Q4_WORD1 quick_sent register	0x005C	R/W
UHCI_REG_Q5_WORD0_REG	Q5_WORD0 quick_sent register	0x0060	R/W
UHCI_REG_Q5_WORD1_REG	Q5_WORD1 quick_sent register	0x0064	R/W
UHCI_REG_Q6_WORD0_REG	Q6_WORD0 quick_sent register	0x0068	R/W
UHCI_REG_Q6_WORD1_REG	Q6_WORD1 quick_sent register	0x006C	R/W
UHCI_ESC_CONFO_REG	Escape sequence configuration register 0	0x0070	R/W
UHCI_ESC_CONF1_REG	Escape sequence configuration register 1	0x0074	R/W
UHCI_ESC_CONF2_REG	Escape sequence configuration register 2	0x0078	R/W
UHCI_ESC_CONF3_REG	Escape sequence configuration register 3	0x007C	R/W
UHCI_PKT_THRES_REG	Configuration register for packet length	0x0080	R/W
UHCI Interrupt Register			
UHCI_INT_RAW_REG	Raw interrupt status	0x0004	varies

Name	Description	Address	Access
UHCI_INT_ST_REG	Masked interrupt status	0x0008	RO
UHCI_INT_ENA_REG	Interrupt enable bits	0x000C	R/W
UHCI_INT_CLR_REG	Interrupt clear bits	0x0010	WT
UHCI_APP_INT_SET_REG	Software interrupt trigger source	0x0014	WT
UHCI Status Register			
UHCI_STATE0_REG	UHCI receive status	0x001C	RO
UHCI_STATE1_REG	UHCI transmit status	0x0020	RO
UHCI_RX_HEAD_REG	UHCI packet header register	0x0030	RO
Version Register			
UHCI_DATE_REG	UHCI version control register	0x0084	R/W

26.7 Registers

26.7.1 UART Registers

The addresses in this section are relative to **UART Controller** base address provided in Table 4.3-3 in Chapter 4 *System and Memory*.

Register 26.1. UART_FIFO_REG (0x0000)

(reserved)																								UART_RXFIFO_RD_BYTE									
31																								8	7	0							
0 0																								0								Reset	

UART_RXFIFO_RD_BYTE UART n accesses FIFO via this field. (RO)

Register 26.2. UART_MEM_CONF_REG (0x0060)

(reserved)					UART_MEM_FORCE_PU UART_MEM_FORCE_PD					UART_RX_TOUT_THRHD										UART_RX_FLOW_THRHD										UART_TX_SIZE					UART_RX_SIZE					(reserved)				
31		29		28	27	26											17	16											7	6			4	3			1	0						
0	0	0	0	0	0	0xa										0x0										0x1										1		0	Reset					

UART_RX_SIZE This field is used to configure the amount of RAM allocated for RX FIFO. The default number is 128 bytes. (R/W)

UART_TX_SIZE This field is used to configure the amount of RAM allocated for TX FIFO. The default number is 128 bytes. (R/W)

UART_RX_FLOW_THRHD This field is used to configure the maximum amount of data bytes that can be received when hardware flow control works. (R/W)

UART_RX_TOUT_THRHD This field is used to configure the threshold time that receiver takes to receive one byte, in the unit of bit time (the time it takes to transfer one bit). The UART_RXFIFO_TOUT_INT interrupt will be triggered when the receiver takes more time to receive one byte with UART_RX_TOUT_EN set to 1. (R/W)

UART_MEM_FORCE_PD Set this bit to force power down UART RAM. (R/W)

UART_MEM_FORCE_PU Set this bit to force power up UART RAM. (R/W)

Register 26.3. UART_INT_RAW_REG (0x0004)

(reserved)																															UART_WAKEUP_INT_RAW UART_AT_CMD_CHAR_DET_INT_RAW UART_RS485_CLASH_INT_RAW UART_RS485_FRM_ERR_INT_RAW UART_RS485_PARITY_ERR_INT_RAW UART_TX_DONE_INT_RAW UART_TX_BRK_IDLE_DONE_INT_RAW UART_GLITCH_DET_INT_RAW UART_SW_XON_INT_RAW UART_SW_XOFF_INT_RAW UART_RXFIFO_TOUT_INT_RAW UART_BRK_DET_INT_RAW UART_CTS_CHG_INT_RAW UART_DSR_CHG_INT_RAW UART_RXFIFO_INT_RAW UART_FRM_ERR_INT_RAW UART_PARITY_ERR_INT_RAW UART_TXFIFO_EMPTY_INT_RAW UART_RXFIFO_FULL_INT_RAW																		
31																				20										19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	Reset												

UART_RXFIFO_FULL_INT_RAW This interrupt raw bit turns to high level when the receiver receives more data than what UART_RXFIFO_FULL_THRHD specifies. (R/WTC/SS)

UART_TXFIFO_EMPTY_INT_RAW This interrupt raw bit turns to high level when the amount of data in TX FIFO is less than what UART_TXFIFO_EMPTY_THRHD specifies. (R/WTC/SS)

UART_PARITY_ERR_INT_RAW This interrupt raw bit turns to high level when the receiver detects a parity error in the data. (R/WTC/SS)

UART_FRM_ERR_INT_RAW This interrupt raw bit turns to high level when the receiver detects a data frame error. (R/WTC/SS)

UART_RXFIFO_OVF_INT_RAW This interrupt raw bit turns to high level when the receiver receives more data than the capacity of RX FIFO. (R/WTC/SS)

UART_DSR_CHG_INT_RAW This interrupt raw bit turns to high level when the receiver detects the edge change of DSRn signal. (R/WTC/SS)

UART_CTS_CHG_INT_RAW This interrupt raw bit turns to high level when the receiver detects the edge change of CTSn signal. (R/WTC/SS)

UART_BRK_DET_INT_RAW This interrupt raw bit turns to high level when the receiver detects a 0 after the stop bit. (R/WTC/SS)

UART_RXFIFO_TOUT_INT_RAW This interrupt raw bit turns to high level when the receiver takes more time than UART_RX_TOUT_THRHD to receive a byte. (R/WTC/SS)

UART_SW_XON_INT_RAW This interrupt raw bit turns to high level when the receiver receives an XON character and UART_SW_FLOW_CON_EN is set to 1. (R/WTC/SS)

UART_SW_XOFF_INT_RAW This interrupt raw bit turns to high level when the receiver receives an XOFF character and UART_SW_FLOW_CON_EN is set to 1. (R/WTC/SS)

UART_GLITCH_DET_INT_RAW This interrupt raw bit turns to high level when the receiver detects a glitch in the middle of a start bit. (R/WTC/SS)

Continued on the next page...

Register 26.3. UART_INT_RAW_REG (0x0004)

Continued from the previous page...

UART_TX_BRK_DONE_INT_RAW This interrupt raw bit turns to high level when the transmitter completes sending NULL characters, after all data in TX FIFO are sent. (R/WTC/SS)

UART_TX_BRK_IDLE_DONE_INT_RAW This interrupt raw bit turns to high level when the transmitter has kept the shortest duration after sending the last data. (R/WTC/SS)

UART_TX_DONE_INT_RAW This interrupt raw bit turns to high level when the transmitter has sent out all data in FIFO. (R/WTC/SS)

UART_RS485_PARITY_ERR_INT_RAW This interrupt raw bit turns to high level when the receiver detects a parity error from the echo of the transmitter in RS485 mode. (R/WTC/SS)

UART_RS485_FRM_ERR_INT_RAW This interrupt raw bit turns to high level when the receiver detects a data frame error from the echo of the transmitter in RS485 mode. (R/WTC/SS)

UART_RS485_CLASH_INT_RAW This interrupt raw bit turns to high level when a collision is detected between the transmitter and the receiver in RS485 mode. (R/WTC/SS)

UART_AT_CMD_CHAR_DET_INT_RAW This interrupt raw bit turns to high level when the receiver detects the configured UART_AT_CMD_CHAR. (R/WTC/SS)

UART_WAKEUP_INT_RAW This interrupt raw bit turns to high level when the input RXD edge changes more times than what (UART_ACTIVE_THRESHOLD + 3) specifies in Light-sleep mode. (R/WTC/SS)

Register 26.4. UART_INT_ST_REG (0x0008)

(reserved)																				UART_WAKEUP_INT_ST UART_AT_CMD_CHAR_DET_INT_ST UART_RS485_CLASH_INT_ST UART_RS485_FRM_ERR_INT_ST UART_TX_DONE_INT_ST UART_TX_BRK_IDLE_INT_ST UART_GLITCH_DET_INT_ST UART_SW_XON_INT_ST UART_SW_XOFF_INT_ST UART_RXFIFO_TOUT_INT_ST UART_CTS_CHG_INT_ST UART_DSR_CHG_INT_ST UART_RXFIFO_OVF_INT_ST UART_FRM_ERR_INT_ST UART_PARITY_ERR_INT_ST UART_TXFIFO_EMPTY_INT_ST UART_RXFIFO_FULL_INT_ST															
31													20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0 0 0 0 0 0 0 0 0 0 0 0													0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset				

UART_RXFIFO_FULL_INT_ST This is the status bit for UART_RXFIFO_FULL_INT when UART_RXFIFO_FULL_INT_ENA is set to 1. (RO)

UART_TXFIFO_EMPTY_INT_ST This is the status bit for UART_TXFIFO_EMPTY_INT when UART_TXFIFO_EMPTY_INT_ENA is set to 1. (RO)

UART_PARITY_ERR_INT_ST This is the status bit for UART_PARITY_ERR_INT when UART_PARITY_ERR_INT_ENA is set to 1. (RO)

UART_FRM_ERR_INT_ST This is the status bit for UART_FRM_ERR_INT when UART_FRM_ERR_INT_ENA is set to 1. (RO)

UART_RXFIFO_OVF_INT_ST This is the status bit for UART_RXFIFO_OVF_INT when UART_RXFIFO_OVF_INT_ENA is set to 1. (RO)

UART_DSR_CHG_INT_ST This is the status bit for UART_DSR_CHG_INT when UART_DSR_CHG_INT_ENA is set to 1. (RO)

UART_CTS_CHG_INT_ST This is the status bit for UART_CTS_CHG_INT when UART_CTS_CHG_INT_ENA is set to 1. (RO)

UART_BRK_DET_INT_ST This is the status bit for UART_BRK_DET_INT when UART_BRK_DET_INT_ENA is set to 1. (RO)

UART_RXFIFO_TOUT_INT_ST This is the status bit for UART_RXFIFO_TOUT_INT when UART_RXFIFO_TOUT_INT_ENA is set to 1. (RO)

UART_SW_XON_INT_ST This is the status bit for UART_SW_XON_INT when UART_SW_XON_INT_ENA is set to 1. (RO)

UART_SW_XOFF_INT_ST This is the status bit for UART_SW_XOFF_INT when UART_SW_XOFF_INT_ENA is set to 1. (RO)

UART_GLITCH_DET_INT_ST This is the status bit for UART_GLITCH_DET_INT when UART_GLITCH_DET_INT_ENA is set to 1. (RO)

UART_TX_BRK_DONE_INT_ST This is the status bit for UART_TX_BRK_DONE_INT when UART_TX_BRK_DONE_INT_ENA is set to 1. (RO)

Continued on the next page...

Register 26.4. UART_INT_ST_REG (0x0008)

Continued from the previous page...

UART_TX_BRK_IDLE_DONE_INT_ST This is the status bit for UART_TX_BRK_IDLE_DONE_INT when UART_TX_BRK_IDLE_DONE_INT_ENA is set to 1. (RO)

UART_TX_DONE_INT_ST This is the status bit for UART_TX_DONE_INT when UART_TX_DONE_INT_ENA is set to 1. (RO)

UART_RS485_PARITY_ERR_INT_ST This is the status bit for UART_RS485_PARITY_ERR_INT when UART_RS485_PARITY_INT_ENA is set to 1. (RO)

UART_RS485_FRM_ERR_INT_ST This is the status bit for UART_RS485_FRM_ERR_INT when UART_RS485_FRM_ERR_INT_ENA is set to 1. (RO)

UART_RS485_CLASH_INT_ST This is the status bit for UART_RS485_CLASH_INT when UART_RS485_CLASH_INT_ENA is set to 1. (RO)

UART_AT_CMD_CHAR_DET_INT_ST This is the status bit for UART_AT_CMD_CHAR_DET_INT when UART_AT_CMD_CHAR_DET_INT_ENA is set to 1. (RO)

UART_WAKEUP_INT_ST This is the status bit for UART_WAKEUP_INT when UART_WAKEUP_INT_ENA is set to 1. (RO)

Register 26.5. UART_INT_ENA_REG (0x000C)

[illegible]

UART_RXFIFO_FULL_INT_ENA This is the enable bit for UART_RXFIFO_FULL_INT. (R/W)

UART_TXFIFO_EMPTY_INT_ENA This is the enable bit for UART_TXFIFO_EMPTY_INT. (R/W)

UART_PARITY_ERR_INT_ENA This is the enable bit for UART_PARITY_ERR_INT. (R/W)

UART_FRM_ERR_INT_ENA This is the enable bit for UART_FRM_ERR_INT. (R/W)

UART_RXFIFO_OVF_INT_ENA This is the enable bit for UART_RXFIFO_OVF_INT. (R/W)

UART_DSR_CHG_INT_ENA This is the enable bit for UART_DSR_CHG_INT. (R/W)

UART_CTS_CHG_INT_ENA This is the enable bit for UART_CTS_CHG_INT. (R/W)

UART_BRK_DET_INT_ENA This is the enable bit for UART_BRK_DET_INT. (R/W)

UART_RXFIFO_TOUT_INT_ENA This is the enable bit for UART_RXFIFO_TOUT_INT. (R/W)

UART_SW_XON_INT_ENA This is the enable bit for UART_SW_XON_INT. (R/W)

UART_SW_XOFF_INT_ENA This is the enable bit for UART_SW_XOFF_INT. (R/W)

UART_GLITCH_DET_INT_ENA This is the enable bit for UART_GLITCH_DET_INT. (R/W)

UART_TX_BRK_DONE_INT_ENA This is the enable bit for UART_TX_BRK_DONE_INT. (R/W)

UART_TX_BRK_IDLE_DONE_INT_ENA This is the enable bit for UART_TX_BRK_IDLE_DONE_INT.
(R/W)

UART_TX_DONE_INT_ENA This is the enable bit for UART_TX_DONE_INT. (R/W)

Continued on the next page...

Register 26.5. UART_INT_ENA_REG (0x000C)

Continued from the previous page...

UART_RS485_PARITY_ERR_INT_ENA This is the enable bit for UART_RS485_PARITY_ERR_INT.
(R/W)

UART_RS485_FRM_ERR_INT_ENA This is the enable bit for UART_RS485_PARITY_ERR_INT. (R/W)

UART_RS485_CLASH_INT_ENA This is the enable bit for UART_RS485_CLASH_INT. (R/W)

UART_AT_CMD_CHAR_DET_INT_ENA This is the enable bit for UART_AT_CMD_CHAR_DET_INT.
(R/W)

UART_WAKEUP_INT_ENA This is the enable bit for UART_WAKEUP_INT. (R/W)

Register 26.6. UART_INT_CLR_REG (0x0010)

(reserved)																UART_WAKEUP_INT_CLR UART_AT_CMD_CHAR_DET_INT_CLR UART_RS485_CLASH_INT_CLR UART_RS485_FRM_ERR_INT_CLR UART_RS485_PARITY_ERR_INT_CLR UART_TX_DONE_INT_CLR UART_TX_BRK_IDLE_DONE_INT_CLR UART_GLITCH_DET_INT_CLR UART_SW_XOFF_INT_CLR UART_SW_XON_INT_CLR UART_RXFIFO_TOUT_INT_CLR UART_BRK_DET_INT_CLR UART_CTS_CHG_INT_CLR UART_DSR_CHG_INT_CLR UART_RXFIFO_OVF_INT_CLR UART_FRM_ERR_INT_CLR UART_PARITY_ERR_INT_CLR UART_TXFIFO_EMPTY_INT_CLR UART_RXFIFO_FULL_INT_CLR																
31												20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset		

UART_RXFIFO_FULL_INT_CLR Set this bit to clear the UART_THE_RXFIFO_FULL_INT interrupt. (WT)

UART_TXFIFO_EMPTY_INT_CLR Set this bit to clear the UART_TXFIFO_EMPTY_INT interrupt. (WT)

UART_PARITY_ERR_INT_CLR Set this bit to clear the UART_PARITY_ERR_INT interrupt. (WT)

UART_FRM_ERR_INT_CLR Set this bit to clear the UART_FRM_ERR_INT interrupt. (WT)

UART_RXFIFO_OVF_INT_CLR Set this bit to clear the UART_UART_RXFIFO_OVF_INT interrupt. (WT)

UART_DSR_CHG_INT_CLR Set this bit to clear the UART_DSR_CHG_INT interrupt. (WT)

UART_CTS_CHG_INT_CLR Set this bit to clear the UART_CTS_CHG_INT interrupt. (WT)

UART_BRK_DET_INT_CLR Set this bit to clear the UART_BRK_DET_INT interrupt. (WT)

UART_RXFIFO_TOUT_INT_CLR Set this bit to clear the UART_RXFIFO_TOUT_INT interrupt. (WT)

UART_SW_XON_INT_CLR Set this bit to clear the UART_SW_XON_INT interrupt. (WT)

UART_SW_XOFF_INT_CLR Set this bit to clear the UART_SW_XOFF_INT interrupt. (WT)

UART_GLITCH_DET_INT_CLR Set this bit to clear the UART_GLITCH_DET_INT interrupt. (WT)

UART_TX_BRK_DONE_INT_CLR Set this bit to clear the UART_TX_BRK_DONE_INT interrupt. (WT)

UART_TX_BRK_IDLE_DONE_INT_CLR Set this bit to clear the UART_TX_BRK_IDLE_DONE_INT interrupt. (WT)

UART_TX_DONE_INT_CLR Set this bit to clear the UART_TX_DONE_INT interrupt. (WT)

UART_RS485_PARITY_ERR_INT_CLR Set this bit to clear the UART_RS485_PARITY_ERR_INT interrupt. (WT)

Continued on the next page...

Register 26.6. UART_INT_CLR_REG (0x0010)

Continued from the previous page...

UART_RS485_FRM_ERR_INT_CLR Set this bit to clear the UART_RS485_FRM_ERR_INT interrupt. (WT)

UART_RS485_CLASH_INT_CLR Set this bit to clear the UART_RS485_CLASH_INT interrupt. (WT)

UART_AT_CMD_CHAR_DET_INT_CLR Set this bit to clear the UART_AT_CMD_CHAR_DET_INT interrupt. (WT)

UART_WAKEUP_INT_CLR Set this bit to clear the UART_WAKEUP_INT interrupt. (WT)

Register 26.7. UART_CLKDIV_REG (0x0014)

(reserved)								UART_CLKDIV_FRAG												(reserved)								UART_CLKDIV															
31								24	23	20	19								12	11											0												
0	0	0	0	0	0	0	0	0x0		0	0	0	0	0	0	0	0	0x2b6												Reset													

UART_CLKDIV The integral part of the frequency divisor. (R/W)

UART_CLKDIV_FRAG The fractional part of the frequency divisor. (R/W)

Register 26.8. UART_RX_FILT_REG (0x0018)

(reserved)																								UART_GLITCH_FILT_EN		UART_GLITCH_FILT							
31																								9	8	7	0						
0 0																								0	0x8							Reset	

UART_GLITCH_FILT When input pulse width is lower than this value, the pulse is ignored. (R/W)

UART_GLITCH_FILT_EN Set this bit to enable RX signal filter. (R/W)

Register 26.9. UART_CONFO_REG (0x0020)

(reserved)			UART_MEM_CLK_EN		UART_AUTOBAUD_EN		UART_ERR_WR_MASK		UART_CLK_EN		UART_DTR_INV		UART_RTS_INV		UART_TXD_INV		UART_DSR_INV		UART_CTS_INV		UART_RXD_INV		UART_TXFIFO_RST		UART_RXFIFO_RST		UART_IRDA_EN		UART_TX_FLOW_EN		UART_LOOPBACK		UART_IRDA_RX_INV		UART_IRDA_TX_INV		UART_IRDA_WCTL		UART_IRDA_TX_DPLX		UART_TXD_BRK		UART_SW_DTR		UART_SW_RTS		UART_STOP_BIT_NUM		UART_BIT_NUM		UART_PARITY_EN		UART_PARITY	
31	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Reset																							
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	3	0	0																	

UART_PARITY This bit is used to configure the parity check mode. (R/W)

UART_PARITY_EN Set this bit to enable UART parity check. (R/W)

UART_BIT_NUM This field is used to set the length of data. (R/W)

UART_STOP_BIT_NUM This field is used to set the length of stop bit. (R/W)

UART_SW_RTS This bit is used to configure the software RTS signal which is used in software flow control. (R/W)

UART_SW_DTR This bit is used to configure the software DTR signal which is used in software flow control. (R/W)

UART_TXD_BRK Set this bit to enable the transmitter to send NULL characters when the process of sending data is done. (R/W)

UART_IRDA_DPLX Set this bit to enable IrDA loopback mode. (R/W)

UART_IRDA_TX_EN This is the start enable bit for IrDA transmitter. (R/W)

UART_IRDA_WCTL 0: Set IrDA transmitter's 11th bit to 0; 1: The IrDA transmitter's 11th bit is the same as 10th bit. (R/W)

UART_IRDA_TX_INV Set this bit to invert the level of IrDA transmitter. (R/W)

UART_IRDA_RX_INV Set this bit to invert the level of IrDA receiver. (R/W)

UART_LOOPBACK Set this bit to enable UART loopback test mode. (R/W)

UART_TX_FLOW_EN Set this bit to enable flow control function for transmitter. (R/W)

UART_IRDA_EN Set this bit to enable IrDA protocol. (R/W)

UART_RXFIFO_RST Set this bit to reset the UART RX FIFO. (R/W)

UART_TXFIFO_RST Set this bit to reset the UART TX FIFO. (R/W)

UART_RXD_INV Set this bit to invert the level value of UART RXD signal. (R/W)

UART_CTS_INV Set this bit to invert the level value of UART CTS signal. (R/W)

UART_DSR_INV Set this bit to invert the level value of UART DSR signal. (R/W)

Continued on the next page...

Register 26.9. UART_CONFO_REG (0x0020)

Continued from the previous page...

UART_TXD_INV Set this bit to invert the level value of UART TXD signal. (R/W)**UART_RTS_INV** Set this bit to invert the level value of UART RTS signal. (R/W)**UART_DTR_INV** Set this bit to invert the level value of UART DTR signal. (R/W)**UART_CLK_EN** 0: Support clock only when application writes registers; 1: Force clock on for registers. (R/W)**UART_ERR_WR_MASK** 0: Receiver stores the data even if the received data is wrong; 1: Receiver stops storing data into FIFO when data is wrong. (R/W)**UART_AUTOBAUD_EN** This is the enable bit for baud rate detection. (R/W)**UART_MEM_CLK_EN** The signal to enable UART RAM clock gating. (R/W)**Register 26.10. UART_CONF1_REG (0x0024)**

(reserved)																UART_RX_TOUT_EN UART_RX_FLOW_EN UART_DIS_RX_DAT_OVF								UART_TXFIFO_EMPTY_THRHD										UART_RXFIFO_FULL_THRHD											
31								24								23	22	21	20	19								10								9	0								
0								0								0	0	0	0	0x60												0x60												Reset	

UART_RXFIFO_FULL_THRHD An UART_RXFIFO_FULL_INT interrupt is generated when the receiver receives more data than the value of this field. (R/W)**UART_TXFIFO_EMPTY_THRHD** An UART_TXFIFO_EMPTY_INT interrupt is generated when the number of data bytes in TX FIFO is less than the value of this field. (R/W)**UART_DIS_RX_DAT_OVF** Disable UART RX data overflow detection. (R/W)**UART_RX_TOUT_FLOW_DIS** Set this bit to stop accumulating idle_cnt when hardware flow control works. (R/W)**UART_RX_FLOW_EN** This is the flow enable bit for UART receiver. (R/W)**UART_RX_TOUT_EN** This is the enable bit for UART receiver's timeout function. (R/W)

Register 26.11. UART_FLOW_CONF_REG (0x0034)

(reserved)																															UART_SEND_XOFF							UART_SEND_XON							UART_FORCE_XOFF							UART_FORCE_XON							UART_XONOFF_DEL							UART_SW_FLOW_CON_EN																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																		
31																															6							5							4							3							2							1							0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																											
0																															0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0							0													

Reset

UART_SW_FLOW_CON_EN Set this bit to enable software flow control. When UART receives flow control characters XON or XOFF, which can be configured by UART_XON_CHAR or UART_XOFF_CHAR respectively, UART_SW_XON_INT or UART_SW_XOFF_INT interrupts can be triggered if enabled. (R/W)

UART_XONOFF_DEL Set this bit to remove flow control characters from the received data. (R/W)

UART_FORCE_XON Set this bit to force the transmitter to send data. (R/W)

UART_FORCE_XOFF Set this bit to stop the transmitter from sending data. (R/W)

UART_SEND_XON Set this bit to send an XON character. This bit is cleared by hardware automatically. (R/W/SS/SC)

UART_SEND_XOFF Set this bit to send an XOFF character. This bit is cleared by hardware automatically. (R/W/SS/SC)

Register 26.12. UART_SLEEP_CONF_REG (0x0038)

(reserved)																UART_ACTIVE_THRESHOLD																															
31																9																0															
0 0																0xf0																Reset															

Reset

UART_ACTIVE_THRESHOLD UART is activated from Light-sleep mode when the input RXD edge changes more times than the value of this field plus 3. (R/W)

Register 26.13. UART_SWFC_CONFO_REG (0x003C)

(reserved)																UART_XOFF_CHAR				UART_XOFF_THRESHOLD				
3118																1710				90				
0000000000000000																0x13				0xe0				Reset

UART_XOFF_THRESHOLD When the number of data bytes in RX FIFO is more than the value of this field with UART_SW_FLOW_CON_EN set to 1, the transmitter sends an XOFF character. (R/W)

UART_XOFF_CHAR This field stores the XOFF flow control character. (R/W)

Register 26.14. UART_SWFC_CONF1_REG (0x0040)

(reserved)																UART_XON_CHAR				UART_XON_THRESHOLD				
3118																1710				90				
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																0x11				0x0				Reset

UART_XON_THRESHOLD When the number of data bytes in RX FIFO is less than the value of this field with UART_SW_FLOW_CON_EN set to 1, the transmitter sends an XON character. (R/W)

UART_XON_CHAR This field stores the XON flow control character. (R/W)

Register 26.15. UART_TXBRK_CONF_REG (0x0044)

(reserved)																UART_TX_BRK_NUM					
31																8	7	0			
0 0																0xa				Reset	

UART_TX_BRK_NUM This field is used to configure the number of 0 to be sent after the process of sending data is done. It is active when UART_TXD_BRK is set to 1. (R/W)

Register 26.16. UART_IDLE_CONF_REG (0x0048)

Register structure diagram for UART_TX_IDLE_NUM and UART_RX_IDLE_THRHD. The register is 32 bits wide, divided into three fields: (reserved) [31:20], UART_TX_IDLE_NUM [19:10], and UART_RX_IDLE_THRHD [9:0]. The UART_TX_IDLE_NUM field is initialized to 0x100, and the UART_RX_IDLE_THRHD field is initialized to 0x100. A 'Reset' label is shown at the bottom right.

UART_RX_IDLE_THRHD A frame end signal is generated when the receiver takes more time to receive one byte data than the value of this field, in the unit of bit time (the time it takes to transfer one bit). (R/W)

UART_TX_IDLE_NUM This field is used to configure the duration time between transfers, in the unit of bit time (the time it takes to transfer one bit). (R/W)

Register 26.17. UART_RS485_CONF_REG (0x004C)

[illegible]

UART_RS485_EN Set this bit to choose RS485 mode. (R/W)

UART_DLO_EN Configures whether or not to add a turnaround delay of 1 bit before the start bit.

0: Not add

1: Add

(R/W)

UART_DL1_EN Configures whether or not to add a turnaround delay of 1 bit after the stop bit.

0: Not add

1: Add

(R/W)

UART_RS485TX_RX_EN Set this bit to enable receiver could receive data when the transmitter is transmitting data in RS485 mode. (R/W)

UART_RS485RXBY_TX_EN 1: enable RS485 transmitter to send data when RS485 receiver line is busy. (R/W)

UART_RS485_RX_DLY_NUM This bit is used to delay the receiver's internal data signal. (R/W)

UART RS485 TX DLY NUM This field is used to delay the transmitter's internal data signal. (R/W)

Register 26.18. UART_CLK_CONF_REG (0x0078)

(reserved)				UART_RX_RST_CORE		UART_TX_RST_CORE		UART_RX_SCLK_EN		UART_TX_SCLK_EN		UART_RST_CORE		UART_SCLK_EN		UART_SCLK_SEL				UART_SCLK_DIV_NUM				UART_SCLK_DIV_A				UART_SCLK_DIV_B			
31	28	27	26	25	24	23	22	21	20	19	12				11	6				5	0										
0	0	0	0	0	0	1	1	0	1	3	0x1				0x0				0x0				0x0				Reset				

Reset

UART_SCLK_DIV_B The denominator of the frequency divisor. (R/W)**UART_SCLK_DIV_A** The numerator of the frequency divisor. (R/W)**UART_SCLK_DIV_NUM** The integral part of the frequency divisor. (R/W)**UART_SCLK_SEL** Selects UART clock source. 1: APB_CLK; 2: RC_FAST_CLK; 3: XTAL_CLK. (R/W)**UART_SCLK_EN** Set this bit to enable UART TX/RX clock. (R/W)**UART_RST_CORE** Write 1 and then write 0 to this bit, to reset UART TX/RX. (R/W)**UART_TX_SCLK_EN** Set this bit to enable UART TX clock. (R/W)**UART_RX_SCLK_EN** Set this bit to enable UART RX clock. (R/W)**UART_TX_RST_CORE** Write 1 and then write 0 to this bit, to reset UART TX. (R/W)**UART_RX_RST_CORE** Write 1 and then write 0 to this bit, to reset UART RX. (R/W)**Register 26.19. UART_STATUS_REG (0x001C)**

UART_TXD				UART_RTSN				UART_DTRN				(reserved)				UART_TXFIFO_CNT																UART_RXD				UART_CTSN				UART_DSRN				(reserved)				UART_RXFIFO_CNT															
31	30	29	28	26				25	16																15	14	13	12	10				9	0																													
1	1	1	0	0	0	0																1	1	0	0	0	0	0																Reset																			

Reset

UART_RXFIFO_CNT Stores the number of valid data bytes in RX FIFO. (RO)**UART_DSRN** This bit represents the level of the internal UART DSR signal. (RO)**UART_CTSN** This bit represents the level of the internal UART CTS signal. (RO)**UART_RXD** This bit represents the level of the internal UART RXD signal. (RO)**UART_TXFIFO_CNT** Stores the number of data bytes in TX FIFO. (RO)**UART_DTRN** This bit represents the level of the internal UART DTR signal. (RO)**UART_RTSN** This bit represents the level of the internal UART RTS signal. (RO)**UART_TXD** This bit represents the level of the internal UART TXD signal. (RO)

Register 26.20. UART_MEM_TX_STATUS_REG (0x0064)

(reserved)												UART_TX_RADDR												(reserved)			UART_APB_TX_WADDR												Reset
31 21 20												11 10 9												0															
0 0 0 0 0 0 0 0 0 0 0 0												0x0												0			0x0												

UART_APB_TX_WADDR This field stores the offset address in TX FIFO when software writes TX FIFO via APB. (RO)

UART_TX_RADDR This field stores the offset address in TX FIFO when TX FSM reads data via Tx_FIFO_Ctrl. (RO)

Register 26.21. UART_MEM_RX_STATUS_REG (0x0068)

(reserved)												UART_RX_WADDR												(reserved)			UART_APB_RX_RADDR																																				
31												20												11												10			9												0												
0 0 0 0 0 0 0 0 0 0 0 0												0x200												0			0x200												Reset																								

UART_APB_RX_RADDR This field stores the offset address in RX FIFO when software reads data from RX FIFO via APB. UART0 is 0x200. UART1 is 0x280. UART2 is 0x300. (RO)

UART_RX_WADDR This field stores the offset address in RX FIFO when Rx_FIFO_Ctrl writes RX FIFO. UART0 is 0x200. UART1 is 0x280. UART2 is 0x300. (RO)

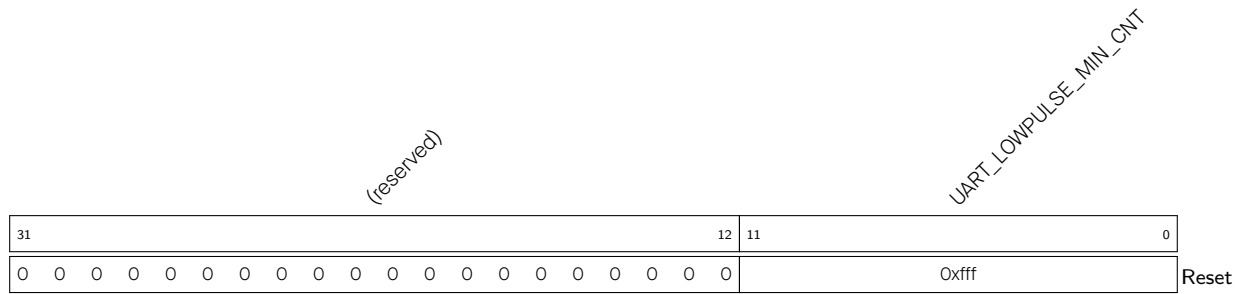
Register 26.22. UART_FSM_STATUS_REG (0x006C)

(reserved)																												UART_ST_UTX_OUT								UART_ST_URX_OUT																							
31																												7								4								3								0							
0 0																												0								0								Reset															

UART_ST_URX_OUT This is the status field of the receiver. (RO)

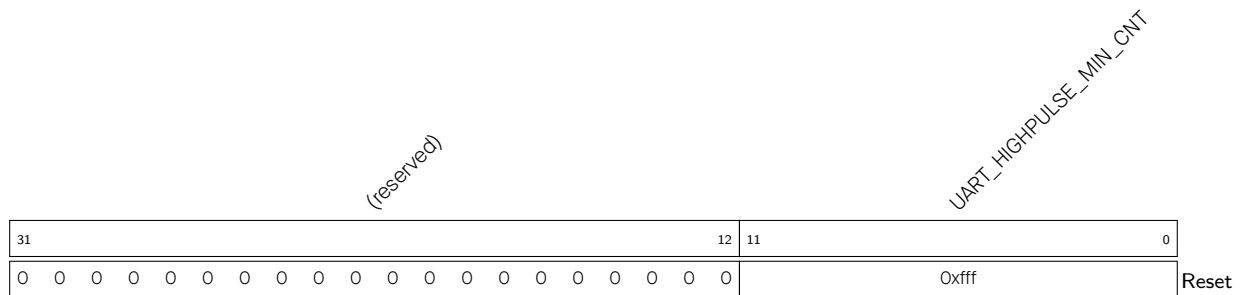
UART_ST_UTX_OUT This is the status field of the transmitter. (RO)

Register 26.23. UART_LOWPULSE_REG (0x0028)



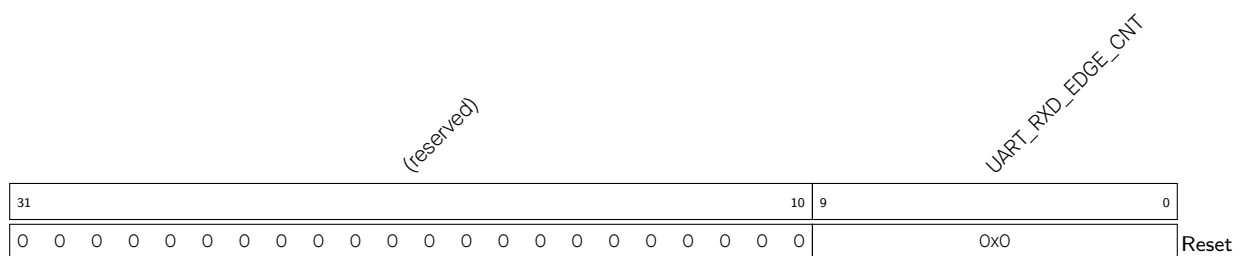
UART_LOWPULSE_MIN_CNT This field stores the value of the minimum duration time of the low level pulse, in the unit of APB_CLK cycles. It is used in baud rate detection. (RO)

Register 26.24. UART_HIGHPULSE_REG (0x002C)



UART_HIGHPULSE_MIN_CNT This field stores the value of the maximum duration time for the high level pulse, in the unit of APB_CLK cycles. It is used in baud rate detection. (RO)

Register 26.25. UART_RXD_CNT_REG (0x0030)



UART_RXD_EDGE_CNT This field stores the count of RXD edge change. It is used in baud rate detection. (RO)

Register 26.26. UART_POSPULSE_REG (0x0070)

(reserved)																UART_POSEDGE_MIN_CNT																																															
31																12																11																0															
0 0																0xffff																Reset																															

UART_POSEDGE_MIN_CNT This field stores the minimal input clock count between two positive edges. It is used in baud rate detection. (RO)

Register 26.27. UART_NEGPULSE_REG (0x0074)

(reserved)																								UART_NEGEDGE_MIN_CNT																							
31																								12								11								0							
0 0																								0fff								Reset															

UART_NEGEDGE_MIN_CNT This field stores the minimal input clock count between two negative edges. It is used in baud rate detection. (RO)

Register 26.28. UART_AT_CMD_PRECNT_REG (0x0050)

(reserved)																UART_PRE_IDLE_NUM																															
31																15																0															
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																0x901																Reset															

UART_PRE_IDLE_NUM This field is used to configure the idle duration time before the first AT_CMD is received by the receiver, in the unit of bit time (the time it takes to transfer one bit). (R/W)

Register 26.29. UART_AT_CMD_POSTCNT_REG (0x0054)

(reserved)																UART_POST_IDLE_NUM															
31																0															
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																0x901															

Reset

UART_POST_IDLE_NUM This field is used to configure the duration time between the last AT_CMD and the next data byte, in the unit of bit time (the time it takes to transfer one bit). (R/W)

Register 26.30. UART_AT_CMD_GAPTOOUT_REG (0x0058)

Diagram illustrating the structure of the `UART_RX_GAP_TOUT` register. The register is 32 bits wide, divided into two 16-bit halves. The top half is labeled `(reserved)` and the bottom half is labeled `UART_RX_GAP_TOUT`. The bottom half contains 11 bits of data, followed by a `Reset` label.

UART_RX_GAP_TOUT This field is used to configure the duration time between the AT_CMD characters, in the unit of bit time (the time it takes to transfer one bit). (R/W)

Register 26.31. UART_AT_CMD_CHAR_REG (0x005C)

(reserved)																UART_CHAR_NUM								UART_AT_CMD_CHAR															
31																15								7								0							
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																0x3								0x2b								Reset							

UART_AT_CMD_CHAR This field is used to configure the content of AT_CMD character. (R/W)

UART_CHAR_NUM This field is used to configure the number of continuous AT_CMD characters received by the receiver. (R/W)

Register 26.32. UART_DATE_REG (0x007C)

UART_DATE	
31	0
0x2008270	
Reset	

UART_DATE This is the version control register. (R/W)

Register 26.33. UART_ID_REG (0x0080)

UART_REG_UPDATE		UART_ID	
UART_UPDATE_CTRL			
31	30	29	0
0	1	0x000500	
Reset			

UART_ID This field is used to configure the UART_ID. (R/W)

UART_UPDATE_CTRL This bit is used to control register synchronization mode. 0: After registers are configured, software needs to write 1 to UART_REG_UPDATE to synchronize registers; 1: Registers are automatically synchronized into UART Core’s clock domain. (R/W)

UART_REG_UPDATE When this bit is set to 1 by software, registers are synchronized to UART Core’s clock domain. This bit is cleared by hardware after synchronization is done. (R/W/SC)

26.7.2 UHCI Regsitors

The addresses in this section are relative to **UHCI Controller** base address provided in Table 4.3-3 in Chapter 4 *System and Memory*.

Register 26.34. UHCI_CONFO_REG (0x0000)

(reserved)																UHCI_UART_RX_BRK_EOF_EN UHCI_CLK_EN UHCI_ENCODE_CRC_EN UHCI_LEN_EOF_EN UHCI_UART_IDLE_EOF_EN UHCI_CRC_REC_EN UHCI_HEAD_EN UHCI_SEPER_EN UHCI_UART2_CE UHCI_UART1_CE UHCI_RX_RST UHCI_TX_RST																													
31																13																12	11	10	9	8	7	6	5	4	3	2	1	0	Reset
0																0																0	0	1	1	0	1	1	1	0	0	0	0	0	

UHCI_TX_RST Write 1, then write 0 to this bit to reset decode state machine. (R/W)

UHCI_RX_RST Write 1, then write 0 to this bit to reset encode state machine. (R/W)

UHCI_UART0_CE Set this bit to link up UHCI and UART0. (R/W)

UHCI_UART1_CE Set this bit to link up UHCI and UART1. (R/W)

UHCI_UART2_CE Set this bit to link up UHCI and UART2. (R/W)

UHCI_SEPER_EN Set this bit to separate the data frame using a special character. (R/W)

UHCI_HEAD_EN Set this bit to encode the data packet with a formatting header. (R/W)

UHCI_CRC_REC_EN Set this bit to enable UHCI to receive the 16 bit CRC. (R/W)

UHCI_UART_IDLE_EOF_EN If this bit is set to 1, UHCI will end the payload receiving process when UART has been in idle state. (R/W)

UHCI_LEN_EOF_EN If this bit is set to 1, UHCI decoder stops receiving payload data when the number of received data bytes has reached the specified value. The value is payload length indicated by UHCI packet header when UHCI_HEAD_EN is 1 or the value is configuration value when UHCI_HEAD_EN is 0. If this bit is set to 0, UHCI decoder stops receiving payload data when 0xC0 has been received. (R/W)

UHCI_ENCODE_CRC_EN Set this bit to enable data integrity check by appending a 16 bit CCITT-CRC to end of the payload. (R/W)

UHCI_CLK_EN 0: Support clock only when application writes registers; 1: Force clock on for registers. (R/W)

UHCI_UART_RX_BRK_EOF_EN If this bit is set to 1, UHCI will end payload receive process when NULL frame is received by UART. (R/W)

Register 26.35. UHCI_CONF1_REG (0x0018)

[illegible]

UHCI_CHECK_SUM_EN	This is the enable bit to check header checksum when UHCI receives a data packet. (R/W)
--------------------------	---

UHCI_CHECK_SEQ_EN This is the enable bit to check sequence number when UHCI receives a data packet. (R/W)

UHCI_CRC_DISABLE Set this bit to support CRC calculation. Data Integrity Check Present bit in UHCI packet frame should be 1. (R/W)

UHCI_SAVE_HEAD Set this bit to save the packet header when UHCI receives a data packet. (R/W)

UHCI_TX_CHECK_SUM_RE Set this bit to encode the data packet with a checksum. (R/W)

UHCI_TX_ACK_NUM_RE Set this bit to encode the data packet with an acknowledgment when a reliable packet is to be transmitted. (R/W)

UHCI_WAIT_SW_START The UHCI encoder will jump to ST_SW_WAIT status if this bit is set to 1.
(R/W)

UHCI_SW_START If current UHCI_ENCODE_STATE is ST_SW_WAIT, the UHCI will start to send data packet out when this bit is set to 1. (R/W/SC)

Register 26.36. UHCI_ESCAPE_CONF_REG (0x0024)

(reserved)																								UHCI_RX_13_ESC_EN UHCI_RX_11_ESC_EN UHCI_RX_DB_ESC_EN UHCI_TX_CO_ESC_EN UHCI_TX_13_ESC_EN UHCI_TX_11_ESC_EN UHCI_TX_DB_ESC_EN UHCI_TX_CO_ESC_EN																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																												
31																								8		7	6	5	4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																			
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0</

Reset

UHCI_TX_CO_ESC_EN Set this bit to to decode character 0xC0 when DMA receives data. (R/W)

UHCI_TX_DB_ESC_EN Set this bit to to decode character 0xDB when DMA receives data. (R/W)

UHCI_TX_11_ESC_EN Set this bit to to decode flow control character 0x11 when DMA receives data. (R/W)

UHCI_TX_13_ESC_EN Set this bit to to decode flow control character 0x13 when DMA receives data. (R/W)

UHCI_RX_CO_ESC_EN Set this bit to replace 0xC0 by special characters when DMA sends data. (R/W)

UHCI_RX_DB_ESC_EN Set this bit to replace 0xDB by special characters when DMA sends data. (R/W)

UHCI_RX_11_ESC_EN Set this bit to replace flow control character 0x11 by special characters when DMA sends data. (R/W)

UHCI_RX_13_ESC_EN Set this bit to replace flow control character 0x13 by special characters when DMA sends data. (R/W)

Register 26.37. UHCI_HUNG_CONF_REG (0x0028)

(reserved)																UHCL_RXFIFO_TIMEOUT_ENA				UHCL_RXFIFO_TIMEOUT_SHIFT				UHCL_RXFIFO_TIMEOUT				UHCL_TXFIFO_TIMEOUT_ENA				UHCL_TXFIFO_TIMEOUT_SHIFT				UHCL_TXFIFO_TIMEOUT			
31								24	23	22	20	19								12	11	10	8	7								0							
0	0	0	0	0	0	0	0	1	0		0x10							1	0		0x10							Reset											

UHCI_TXFIFO_TIMEOUT This field stores the timeout value. UHCI will produce the UHCI_TX_HUNG_INT interrupt when DMA takes more time to receive data. (R/W)

UHCI_TXFIFO_TIMEOUT_SHIFT This field is used to configure the maximum tick count. (R/W)

UHCI_TXFIFO_TIMEOUT_ENA This is the enable bit for TX FIFO receive timeout. (R/W)

UHCI_RXFIFO_TIMEOUT This field stores the timeout value. UHCI will produce the UHCI_RX_HUNG_INT interrupt when DMA takes more time to read data from RAM. (R/W)

UHCI_RXFIFO_TIMEOUT_SHIFT This field is used to configure the maximum tick count. (R/W)

UHCI_RXFIFO_TIMEOUT_ENA This is the enable bit for DMA send timeout. (R/W)

Register 26.38. UHCI_ACK_NUM_REG (0x002C)

(reserved)																												UHCL_ACK_NUM_LOAD		UHCL_ACK_NUM	
31																												4	3	2	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0x0	Reset

UHCI_ACK_NUM This is the ACK number used in software flow control. (R/W)

UHCI_ACK_NUM_LOAD Set this bit to 1, and the value configured by UHCI_ACK_NUM would be loaded. (WT)

Register 26.39. UHCI_QUICK_SENT_REG (0x0034)

[illegible]

UHCI_SINGLE_SEND_NUM This field is used to specify single_send mode. (R/W)

UHCI_SINGLE_SEND_EN Set this bit to enable single_send mode to send short packets. (R/W/SC)

UHCI_ALWAYS_SEND_NUM This field is used to specify always_send mode. (R/W)

UHCI_ALWAYS_SEND_EN Set this bit to enable always_send mode to send short packets. (R/W)

Register 26.40. UHCI_REG_Q0_WORD0_REG (0x0038)

UHCI_SEND_GO_WORD

31		0
0x000000		

Reset

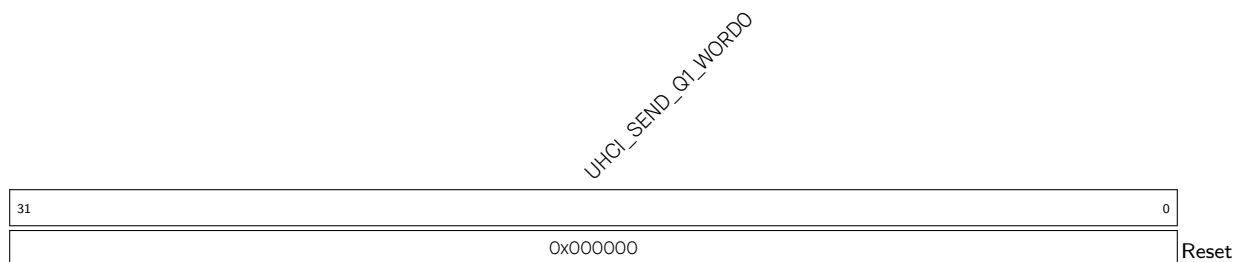
UHCI_SEND_QO_WORD0 This register is used as a quick_sent register when mode is specified by UHCI_ALWAYS_SEND_NUM or UHCI_SINGLE_SEND_NUM. (R/W)

Register 26.41. UHCI_REG_Q0_WORD1_REG (0x003C)

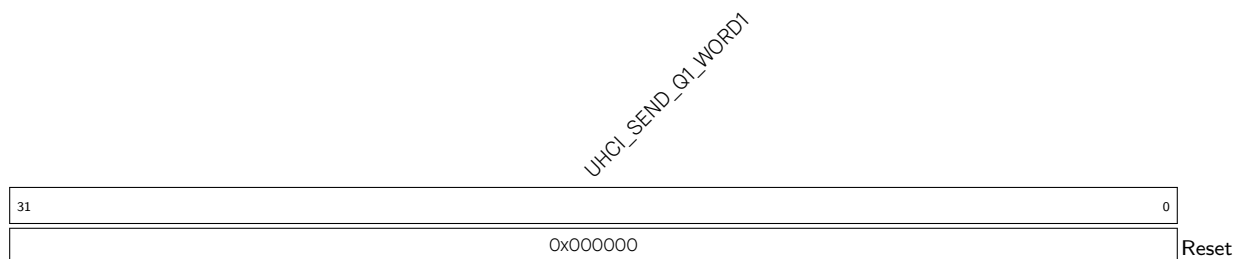
UHCI_SEND_GO_WORD1

31	0
0x000000	
Reset	

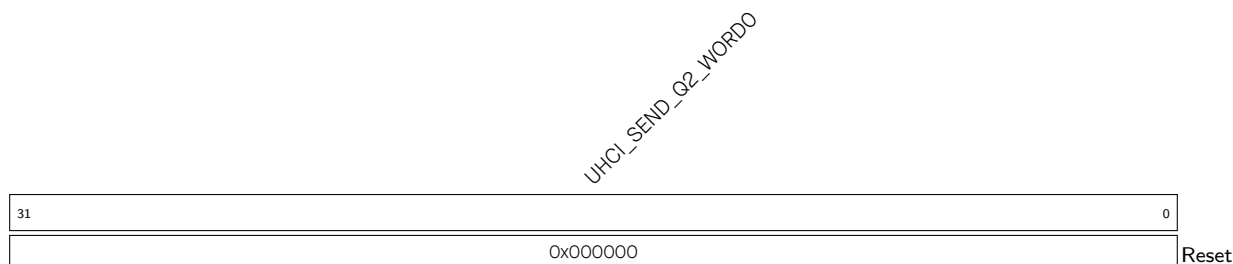
UHCI_SEND_QO_WORD1 This register is used as a quick_sent register when mode is specified by UHCI_ALWAYS_SEND_NUM or UHCI_SINGLE_SEND_NUM. (R/W)

Register 26.42. UHCI_REG_Q1_WORD0_REG (0x0040)

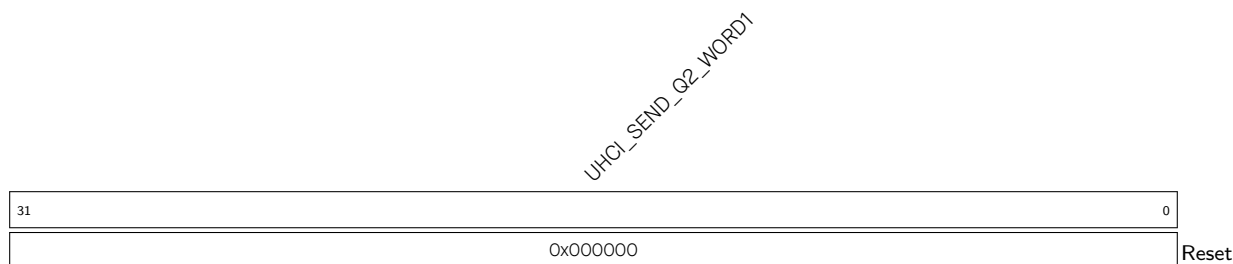
UHCI_SEND_Q1_WORD0 This register is used as a quick_sent register when mode is specified by UHCI_ALWAYS_SEND_NUM or UHCI_SINGLE_SEND_NUM. (R/W)

Register 26.43. UHCI_REG_Q1_WORD1_REG (0x0044)

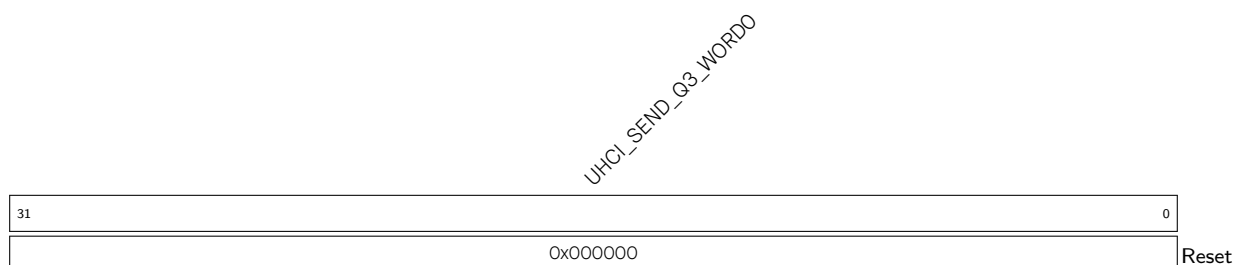
UHCI_SEND_Q1_WORD1 This register is used as a quick_sent register when mode is specified by UHCI_ALWAYS_SEND_NUM or UHCI_SINGLE_SEND_NUM. (R/W)

Register 26.44. UHCI_REG_Q2_WORD0_REG (0x0048)

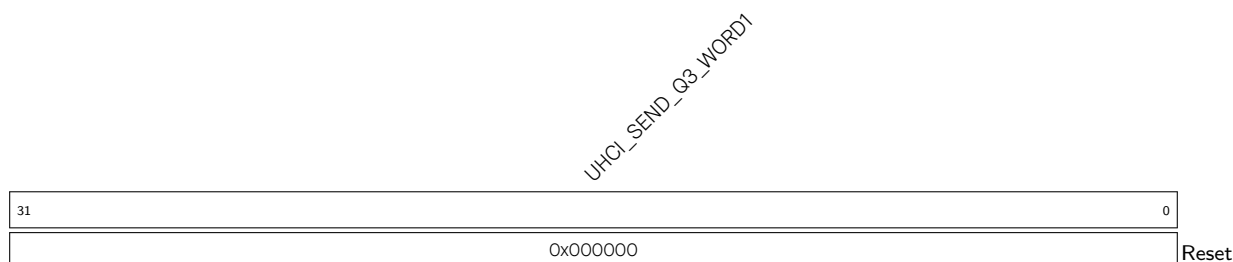
UHCI_SEND_Q2_WORD0 This register is used as a quick_sent register when mode is specified by UHCI_ALWAYS_SEND_NUM or UHCI_SINGLE_SEND_NUM. (R/W)

Register 26.45. UHCI_REG_Q2_WORD1_REG (0x004C)

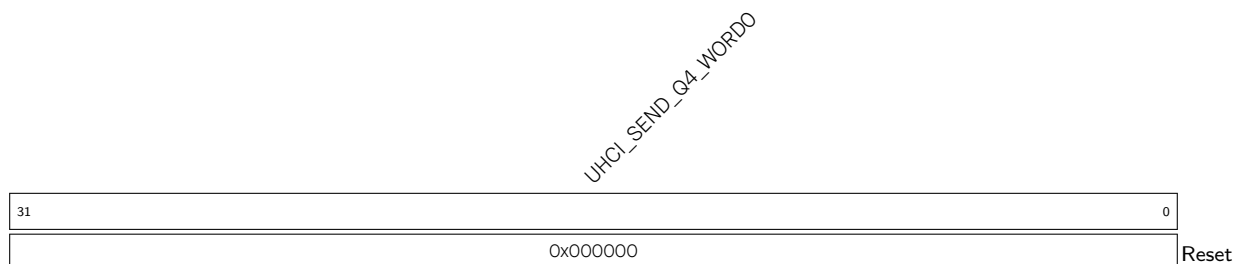
UHCI_SEND_Q2_WORD1 This register is used as a quick_sent register when mode is specified by UHCI_ALWAYS_SEND_NUM or UHCI_SINGLE_SEND_NUM. (R/W)

Register 26.46. UHCI_REG_Q3_WORD0_REG (0x0050)

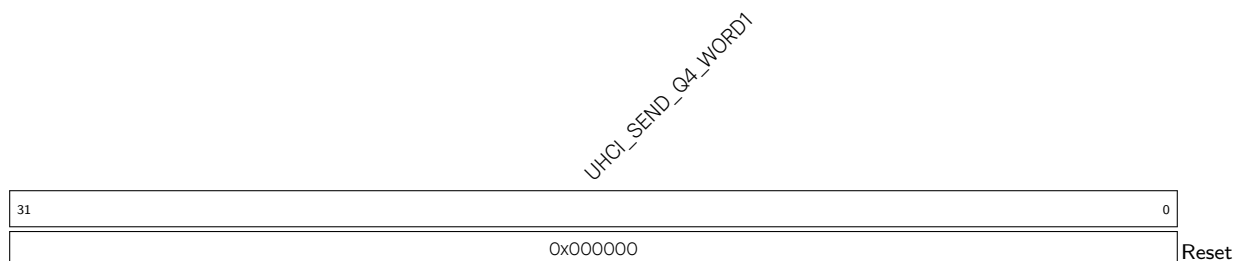
UHCI_SEND_Q3_WORD0 This register is used as a quick_sent register when mode is specified by UHCI_ALWAYS_SEND_NUM or UHCI_SINGLE_SEND_NUM. (R/W)

Register 26.47. UHCI_REG_Q3_WORD1_REG (0x0054)

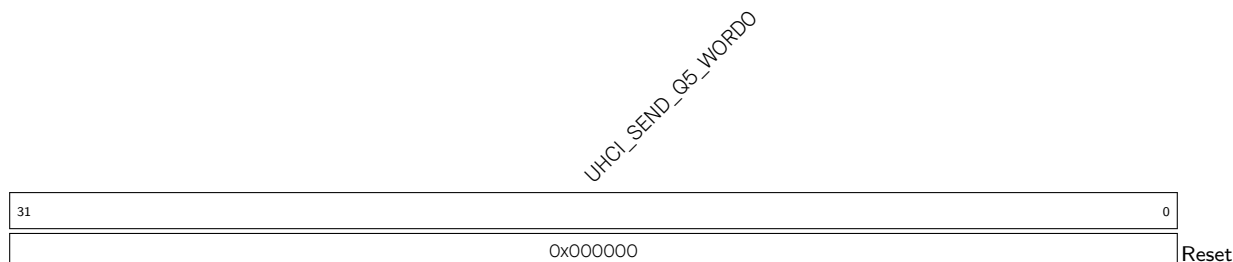
UHCI_SEND_Q3_WORD1 This register is used as a quick_sent register when mode is specified by UHCI_ALWAYS_SEND_NUM or UHCI_SINGLE_SEND_NUM. (R/W)

Register 26.48. UHCI_REG_Q4_WORD0_REG (0x0058)

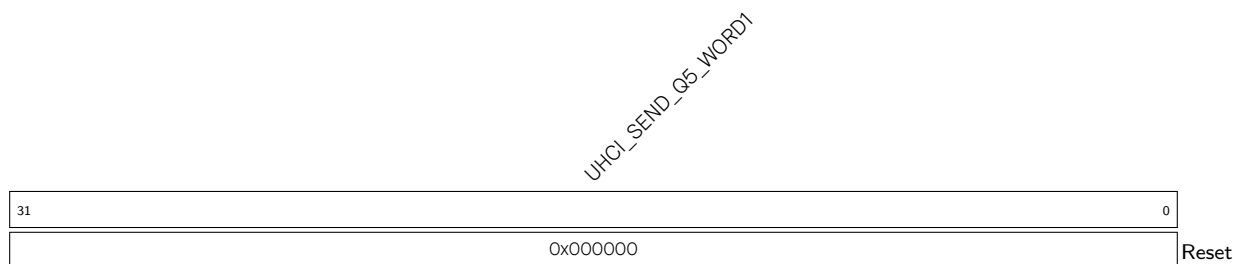
UHCI_SEND_Q4_WORD0 This register is used as a quick_sent register when mode is specified by UHCI_ALWAYS_SEND_NUM or UHCI_SINGLE_SEND_NUM. (R/W)

Register 26.49. UHCI_REG_Q4_WORD1_REG (0x005C)

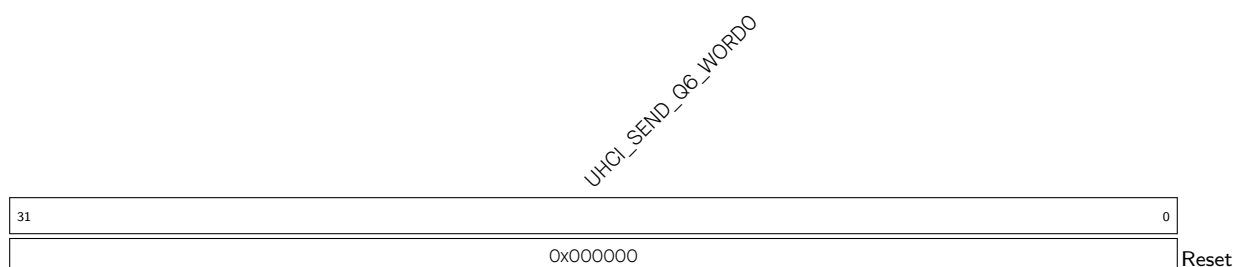
UHCI_SEND_Q4_WORD1 This register is used as a quick_sent register when mode is specified by UHCI_ALWAYS_SEND_NUM or UHCI_SINGLE_SEND_NUM. (R/W)

Register 26.50. UHCI_REG_Q5_WORD0_REG (0x0060)

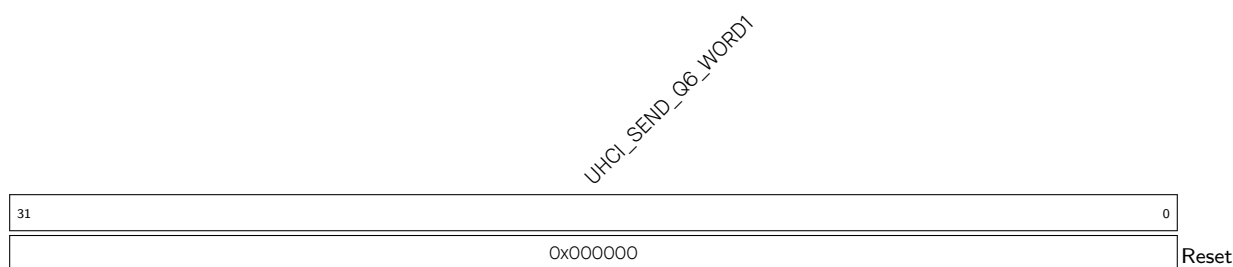
UHCI_SEND_Q5_WORD0 This register is used as a quick_sent register when mode is specified by UHCI_ALWAYS_SEND_NUM or UHCI_SINGLE_SEND_NUM. (R/W)

Register 26.51. UHCI_REG_Q5_WORD1_REG (0x0064)

UHCI_SEND_Q5_WORD1 This register is used as a quick_sent register when mode is specified by UHCI_ALWAYS_SEND_NUM or UHCI_SINGLE_SEND_NUM. (R/W)

Register 26.52. UHCI_REG_Q6_WORD0_REG (0x0068)

UHCI_SEND_Q6_WORD0 This register is used as a quick_sent register when mode is specified by UHCI_ALWAYS_SEND_NUM or UHCI_SINGLE_SEND_NUM. (R/W)

Register 26.53. UHCI_REG_Q6_WORD1_REG (0x006C)

UHCI_SEND_Q6_WORD1 This register is used as a quick_sent register when mode is specified by UHCI_ALWAYS_SEND_NUM or UHCI_SINGLE_SEND_NUM. (R/W)

Register 26.54. UHCI_ESC_CONFO_REG (0x0070)

(reserved)								UHCI_SEPER_ESC_CHAR1								UHCI_SEPER_ESC_CHAR0								UHCI_SEPER_CHAR								
31	24	23	16	15	8	7	0																									
0	0	0	0	0	0	0	0	0xdc								0xdb								0xc0								Reset

UHCI_SEPER_CHAR This field is used to define separators to encode data packets. The default value is 0xc0. (R/W)

UHCI_SEPER_ESC_CHAR0 This field is used to define the first character of SLIP escape sequence. The default value is 0xdb. (R/W)

UHCI_SEPER_ESC_CHAR1 This field is used to define the second character of SLIP escape sequence. The default value is 0xdc. (R/W)

Register 26.55. UHCI_ESC_CONF1_REG (0x0074)

(reserved)								UHCL_ESC_SEQO_CHAR1								UHCL_ESC_SEQO_CHAR0								UHCL_ESC_SEQO								
31	24	23	16	15	8	7	0																									
0	0	0	0	0	0	0	0	0xdd								0xdb								0xdb								Reset

UHCI_ESC_SEQO This field is used to define a character that need to be encoded. The default value is 0xdb that used as the first character of SLIP escape sequence. (R/W)

UHCI_ESC_SEQO_CHAR0 This field is used to define the first character of SLIP escape sequence. The default value is 0xdb. (R/W)

UHCI_ESC_SEQO_CHAR1 This field is used to define the second character of SLIP escape sequence. The default value is 0xdd. (R/W)

Register 26.56. UHCI_ESC_CONF2_REG (0x0078)

(reserved)								UHCI_ESC_SEQ1_CHAR1								UHCI_ESC_SEQ1_CHAR0								UHCI_ESC_SEQ1											
31								24	23								16	15								8	7								0
0	0	0	0	0	0	0	0	Oxde								Oxdb								Ox11								Reset			

UHCI_ESC_SEQ1 This field is used to define a character that need to be encoded. The default value is 0x11 that used as a flow control character. (R/W)

UHCI_ESC_SEQ1_CHAR0 This field is used to define the first character of SLIP escape sequence. The default value is 0xDB. (R/W)

UHCI_ESC_SEQ1_CHAR1 This field is used to define the second character of SLIP escape sequence. The default value is 0xDE. (R/W)

Register 26.57. UHCI_ESC_CONF3_REG (0x007C)

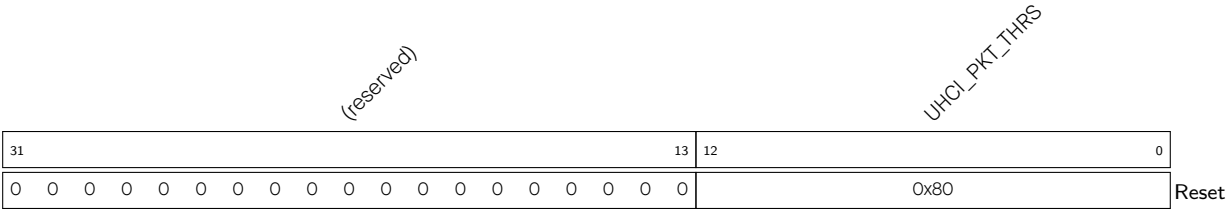
(reserved)								UHCI_ESC_SEQ2_CHAR1								UHCI_ESC_SEQ2_CHAR0								UHCI_ESC_SEQ2											
31								24	23								16	15								8	7								0
0	0	0	0	0	0	0	0	0xdf								0xdb								0x13								Reset			

UHCI_ESC_SEQ2 This field is used to define a character that need to be decoded. The default value is 0x13 that used as a flow control character. (R/W)

UHCI_ESC_SEQ2_CHAR0 This field is used to define the first character of SLIP escape sequence. The default value is 0xDB. (R/W)

UHCI_ESC_SEQ2_CHAR1 This field is used to define the second character of SLIP escape sequence. The default value is 0xDF. (R/W)

Register 26.58. UHCI_PKT_THRES_REG (0x0080)



UHCI_PKT_THRS This field is used to configure the maximum value of the packet length when UHCI_HEAD_EN is 0. (R/W)

Register 26.59. UHCI_INT_RAW_REG (0x0004)

(reserved)																								UHCI_APP_CTRL1_INT_RAW UHCI_APP_CTRL0_INT_RAW UHCI_OUT_EOF_INT_RAW UHCI_SEND_A_REG_Q_INT_RAW UHCI_SEND_S_REG_Q_INT_RAW UHCI_TX_HUNG_INT_RAW UHCI_RX_HUNG_INT_RAW UHCI_TX_START_INT_RAW UHCI_RX_START_INT_RAW																		
31																								9	8	7	6	5	4	3	2	1	0									
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset													

UHCI_RX_START_INT_RAW This is the interrupt raw bit for UHCI_RX_START_INT interrupt. The interrupt is triggered when a separator has been sent. (R/WTC/SS)

UHCI_TX_START_INT_RAW This is the interrupt raw bit for UHCI_TX_START_INT interrupt. The interrupt is triggered when UHCI detects a separator. (R/WTC/SS)

UHCI_RX_HUNG_INT_RAW This is the interrupt raw bit for UHCI_RX_HUNG_INT interrupt. The interrupt is triggered when UHCI takes more time to receive data than configure value. (R/WTC/SS)

UHCI_TX_HUNG_INT_RAW This is the interrupt raw bit for UHCI_TX_HUNG_INT interrupt. The interrupt is triggered when UHCI takes more time to read data from RAM than the configured value. (R/WTC/SS)

UHCI_SEND_S_REG_Q_INT_RAW This is the interrupt raw bit for UHCI_SEND_S_REG_Q_INT interrupt. The interrupt is triggered when UHCI has sent out a short packet using single_send mode. (R/WTC/SS)

UHCI_SEND_A_REG_Q_INT_RAW This is the interrupt raw bit for UHCI_SEND_A_REG_Q_INT interrupt. The interrupt is triggered when UHCI has sent out a short packet using always_send mode. (R/WTC/SS)

UHCI_OUT_EOF_INT_RAW This is the interrupt raw bit for UHCI_OUT_EOF_INT interrupt. The interrupt is triggered when there are some errors in EOF in the transmit descriptors. (R/WTC/SS)

UHCI_APP_CTRL0_INT_RAW This is the interrupt raw bit for UHCI_APP_CTRL0_INT interrupt. The interrupt is triggered when UHCI_APP_CTRL0_IN_SET is set. (R/W)

UHCI_APP_CTRL1_INT_RAW This is the interrupt raw bit for UHCI_APP_CTRL1_INT interrupt. The interrupt is triggered when UHCI_APP_CTRL1_IN_SET is set. (R/W)

Register 26.60. UHCI_INT_ST_REG (0x0008)

(reserved)																								UHCI_APP_CTRL1_INT_ST UHCI_APP_CTRL0_INT_ST UHCI_OUTLINK_EOF_ERR_INT_ST UHCI_SEND_A_REG_Q_INT_ST UHCI_SEND_S_REG_Q_INT_ST UHCI_TX_HUNG_INT_ST UHCI_RX_HUNG_INT_ST UHCI_TX_START_INT_ST UHCI_RX_START_INT_ST										
31																								9	8	7	6	5	4	3	2	1	0	Reset
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0											

UHCI_RX_START_INT_ST This is the masked interrupt bit for UHCI_RX_START_INT interrupt when UHCI_RX_START_INT_ENA is set to 1. (RO)

UHCI_TX_START_INT_ST This is the masked interrupt bit for UHCI_TX_START_INT interrupt when UHCI_TX_START_INT_ENA is set to 1. (RO)

UHCI_RX_HUNG_INT_ST This is the masked interrupt bit for UHCI_RX_HUNG_INT interrupt when UHCI_RX_HUNG_INT_ENA is set to 1. (RO)

UHCI_TX_HUNG_INT_ST This is the masked interrupt bit for UHCI_TX_HUNG_INT interrupt when UHCI_TX_HUNG_INT_ENA is set to 1. (RO)

UHCI_SEND_S_REG_Q_INT_ST This is the masked interrupt bit for UHCI_SEND_S_REG_Q_INT interrupt when UHCI_SEND_S_REG_Q_INT_ENA is set to 1. (RO)

UHCI_SEND_A_REG_Q_INT_ST This is the masked interrupt bit for UHCI_SEND_A_REG_Q_INT interrupt when UHCI_SEND_A_REG_Q_INT_ENA is set to 1. (RO)

UHCI_OUTLINK_EOF_ERR_INT_ST This is the masked interrupt bit for UHCI_OUTLINK_EOF_ERR_INT interrupt when UHCI_OUTLINK_EOF_ERR_INT_ENA is set to 1. (RO)

UHCI_APP_CTRL0_INT_ST This is the masked interrupt bit for UHCI_APP_CTRL0_INT interrupt when UHCI_APP_CTRL0_INT_ENA is set to 1. (RO)

UHCI_APP_CTRL1_INT_ST This is the masked interrupt bit for UHCI_APP_CTRL1_INT interrupt when UHCI_APP_CTRL1_INT_ENA is set to 1. (RO)

Register 26.61. UHCI_INT_ENA_REG (0x000C)

(reserved)																								UHCL_APP_CTRL1_INT_ENA UHCL_APP_CTRL0_INT_ENA UHCL_OUTLINK_EOF_ERR_INT_ENA UHCL_SEND_A_REG_Q_INT_ENA UHCL_SEND_S_REG_Q_INT_ENA UHCL_TX_HUNG_INT_ENA UHCL_RX_HUNG_INT_ENA UHCL_TX_START_INT_ENA UHCL_RX_START_INT_ENA										
31																								9	8	7	6	5	4	3	2	1	0	Reset
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0									

UHCI_RX_START_INT_ENA This is the interrupt enable bit for UHCI_RX_START_INT interrupt. (R/W)

UHCI_TX_START_INT_ENA This is the interrupt enable bit for UHCI_TX_START_INT interrupt. (R/W)

UHCI_RX_HUNG_INT_ENA This is the interrupt enable bit for UHCI_RX_HUNG_INT interrupt. (R/W)

UHCI_TX_HUNG_INT_ENA This is the interrupt enable bit for UHCI_TX_HUNG_INT interrupt. (R/W)

UHCI_SEND_S_REG_Q_INT_ENA This is the interrupt enable bit for UHCI_SEND_S_REG_Q_INT interrupt. (R/W)

UHCI_SEND_A_REG_Q_INT_ENA This is the interrupt enable bit for UHCI_SEND_A_REG_Q_INT interrupt. (R/W)

UHCI_OUTLINK_EOF_ERR_INT_ENA This is the interrupt enable bit for UHCI_OUTLINK_EOF_ERR_INT interrupt. (R/W)

UHCI_APP_CTRL0_INT_ENA This is the interrupt enable bit for UHCI_APP_CTRL0_INT interrupt. (R/W)

UHCI_APP_CTRL1_INT_ENA This is the interrupt enable bit for UHCI_APP_CTRL1_INT interrupt. (R/W)

Register 26.62. UHCI_INT_CLR_REG (0x0010)

(reserved)																								UHCI_APP_CTRL1_INT_CLR UHCI_APP_CTRL0_INT_CLR UHCI_OUTLINK_EOF_ERR_INT_CLR UHCI_SEND_A_REG_Q_INT_CLR UHCI_SEND_S_REG_Q_INT_CLR UHCI_TX_HUNG_INT_CLR UHCI_RX_HUNG_INT_CLR UHCI_TX_START_INT_CLR UHCI_RX_START_INT_CLR										
31																								9	8	7	6	5	4	3	2	1	0	Reset
0 0																								0	0	0	0	0	0	0	0	0	0	

UHCI_RX_START_INT_CLR Set this bit to clear UHCI_RX_START_INT interrupt. (WT)

UHCI_TX_START_INT_CLR Set this bit to clear UHCI_TX_START_INT interrupt. (WT)

UHCI_RX_HUNG_INT_CLR Set this bit to clear UHCI_RX_HUNG_INT interrupt. (WT)

UHCI_TX_HUNG_INT_CLR Set this bit to clear UHCI_TX_HUNG_INT interrupt. (WT)

UHCI_SEND_S_REG_Q_INT_CLR Set this bit to clear UHCI_SEND_S_REG_Q_INT interrupt. (WT)

UHCI_SEND_A_REG_Q_INT_CLR Set this bit to clear UHCI_SEND_A_REG_Q_INT interrupt. (WT)

UHCI_OUTLINK_EOF_ERR_INT_CLR Set this bit to clear UHCI_OUTLINK_EOF_ERR_INT interrupt. (WT)

UHCI_APP_CTRL0_INT_CLR Set this bit to clear UHCI_APP_CTRL0_INT interrupt. (WT)

UHCI_APP_CTRL1_INT_CLR Set this bit to clear UHCI_APP_CTRL1_INT interrupt. (WT)

Register 26.63. UHCI_APP_INT_SET_REG (0x0014)

(reserved)																UHCI_APP_CTRL1_INT_SET UHCI_APP_CTRL0_INT_SET				
31																	2	1	0	Reset
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

UHCI_APP_CTRL0_INT_SET This bit is software interrupt trigger source of UHCI_APP_CTRL0_INT. (WT)

UHCI_APP_CTRL1_INT_SET This bit is software interrupt trigger source of UHCI_APP_CTRL1_INT. (WT)

Register 26.64. UHCI_STATE0_REG (0x001C)

(reserved)																										UHCI_DECODE_STATE		UHCI_RX_ERR_CAUSE		
31																									6	5	3	2	0	
0 0																										0		0		Reset

UHCI_RX_ERR_CAUSE This field indicates the error type when DMA has received a packet with error. 3'b001: Checksum error in the HCI packet. 3'b010: Sequence number error in the HCI packet. 3'b011: CRC bit error in the HCI packet. 3'b100: 0xC0 is found but the received HCI packet is not end. 3'b101: 0xC0 is not found when the HCI packet has been received. 3'b110: CRC check error. (RO)

UHCI_DECODE_STATE UHCI decoder status. (RO)

Register 26.65. UHCI_STATE1_REG (0x0020)

(reserved)																												UHCI_ENCODE_STATE		
31																												3	2	0
0 0																												0		Reset

UHCI_ENCODE_STATE UHCI encoder status. (RO)

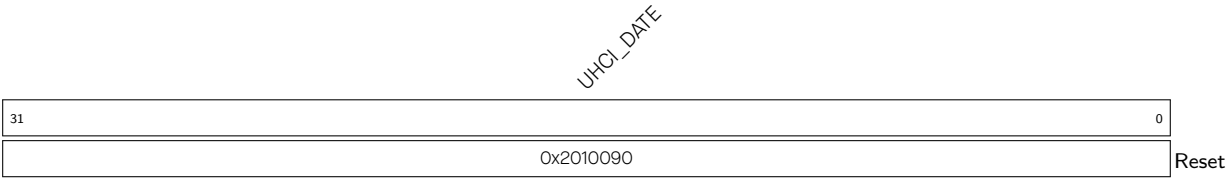
Register 26.66. UHCI_RX_HEAD_REG (0x0030)

UHCI_RX_HEAD																														0
0x000000																														

Reset

UHCI_RX_HEAD This register stores the header of the current received packet. (RO)

Register 26.67. UHCI_DATE_REG (0x0084)



UHCI_DATE This is the version control register. (R/W)